

Bachelor of Technology Capstone Project

DEPARTMENT: ICT

PROGRAM: INFORMATION TECHNOLOGY RQF

LEVEL: 8

**Project Topic: Design and implement a Real Estate Booking
System Using Smart Contract**

Case Study: Kigali

Submitted by: UKUNDAYEZU Hermogene

Reg. No.: 25RP18177

May 2026, Musanze

Supervisor: UWIZEYE Samuel

Co-supervisor: MBABAZI Mary

APPROVAL

This project work was written, arranged, and compiled by **UKUNDAYEZU Hermogene** under the supervision of **Mr. UWIZEYE Samuel** for the award of a bachelor's degree in information technology.

Project Topic: Design and implement a Real Estate Booking System Using Smart Contract

Case Study: Kigali

Mr. UWIZEYE Samuel (*Supervisor*)

Date & Signature: _____

Ms. MBABAZI Mary (*Head of Department*)

Date & Signature: _____

DECLARATION

I, UKUNDAYEZU Hermogene, hereby declare that this capstone project titled "Real Estate Booking System Using Smart Contract: A Case Study of Kigali" is my own original work. It has not been previously submitted for any degree or examination at any other institution. All sources of information used in this work have been duly acknowledged by means of complete references.

I further declare that this project does not contain any material that is defamatory or infringes upon any copyright or other intellectual property right of any person or entity.

Name: **UKUNDAYEZU Hermogene**

Reg. No.: **25RP18177**

Signature: _____

Date: _____

ACKNOWLEDGEMENTS

May God bless you all.

Finally, I wish to express my profound gratitude to my family and friends for their patience, understanding, and moral support throughout my academic journey. Their encouragement has been my driving force.

I am thankful to all the respondents who participated in this study, particularly the tenants, landlords, and real estate agents in Kigali who shared their experiences and insights into the current rental market challenges.

My sincere appreciation goes to the Department of ICT and the entire Rwanda Polytechnic fraternity for providing a conducive learning environment and the necessary resources to complete this project.

I also extend my heartfelt thanks to my co-supervisor, Ms. Mary Mbabazi, for her insightful comments and suggestions that greatly improved the quality of this research.

I am deeply grateful to my supervisor, Mr. UWIZEYE Samuel, for his invaluable guidance, constructive feedback, and unwavering support during the entire capstone project. His expertise and encouragement were instrumental in shaping this work.

First and foremost, I would like to express my sincere gratitude to Almighty God for His grace and guidance throughout this research journey.

LIST OF FIGURES

Figure 1: Conceptual Framework of the Decentralized Escrow System	11
Figure 2: Agile Software Development Methodology	19
Figure 3: A class diagram defines the static structure of the system	23
Figure 4: A sequence diagram maps the dynamic, chronological workflow of the system.....	25
Figure 5: Context Diagram Level 0 DFD — The platform is a single central process interacting with four external entities.....	28
Figure 6: The Level 1 data flow diagram	29
Figure 7 : Use Case Diagram	30
Figure 8: entity relationship diagram	31
Figure 9: physical model	32
Figure 10: Login Page	42
Figure 11: Create Account page.....	43
Figure 12: My Dashboard Tenant Page.....	43
Figure 13: Property Listing Page (Logged In)	44
Figure 14: Property Detail and Booking Page	44
Figure 15: User Dashboard	45
Figure 16: Admin Dashboard.....	45
Figure 17: Bookings Management Page	46
Figure 18: RealEstateEscrow Escrow Contract for Kigali Real Estate Booking Platform	51
Figure 19: Full database schema - 30 Prisma models across 6 enums.....	53
Figure 20: KIGALI REAL ESTATE BOOKING PLATFORM - ARCHITECTURE & STATE MACHINE	59

LIST OF TABLES

Table 1: Architecture Layer Summary	26
Table 2: Infrastructure Stack	36
Table 3: Smart Contract Deployment and verified through an on-chain escrow submission.....	37
Table 4: API Surface	38
Table 5: Payment Flows 3 Methods	39
Table 6: multi-layered security covering authentication, authorization, validation, and blockchain protections.	40
Table 7: Background Jobs	41
Table 8: Verification Evidence The following commands confirm system operation.....	42
Table 9: Authentication & User Management Handles account creation, login, security, and user profiles.....	54
Table 10: Property Management Core property listing and management operations.....	54
Table 11: Booking System Main booking workflow for tenants and owners.....	54
Table 12: Payment System Handles blockchain, mobile money, and card payments	54
Table 13: KYC Verification Identity verification for platform security.	55
Table 14: Messaging & Communication Essential communication between users.....	55
Table 15: Reviews & Ratings Tenant feedback system	55
Table 16: Search & Discovery Property searching and recommendations.	55
Table 17: Wishlist System allows users to save Favorite properties.	56
Table 18: User Administration	56
Table 19: Platform Monitoring	56

Table of Contents

APPROVAL	i
DECLARATION	ii
ACKNOWLEDGEMENTS	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ABBREVIATIONS	ix
ABSTRACT	x
CHAPTER ONE: INTRODUCTION	1
1.1. Background of the study	1
1.2. Statement of the problem	2
1.3. Research Objective	3
1.3.1. General Objective	3
1.3.2. Specific Objectives	3
1.4. Research Questions	4
1.5. Significance	4
1.6. Scope of the Study	4
1.7. Limitations of the Study	5
CHAPTER TWO: LITERATURE REVIEW	6
2.0. Introduction	6
2.1. Definition of Key Concepts	6
2.2. Review of Related Literatures	7
2.2.1. Vulnerabilities in Traditional Real Estate Intermediation	7
2.2.2. Blockchain and Smart Contracts as Escrow Mechanisms	7
2.2.3. Decentralized Applications (dApps) in PropTech	8
2.3. Theoretical review	8
2.3.1. Agency Theory (The Principal-Agent Problem)	8
2.3.2. Transaction Cost Economics (TCE)	9
2.4. Empirical Review	9
2.4.1. Global Implementations of Blockchain in Real Estate	9
2.4.2. Empirical Realities of the Kigali Housing Market	10
2.4.3. Comparison and Orientation of Novelty	10
2.5. Conceptual Framework	11
2.6. Conclusion	12

CHAPTER THREE: METHODOLOGY	13
3.0. Introduction	13
3.1. Research Design.....	13
3.2. Sampling Strategies.....	13
3.3. Data Collection Methods	14
3.4. Data Analysis Methods	14
3.5. Data Quality Control	14
3.6. System Analysis Methods	15
3.7. System Design and Development Methods	15
3.7.1. Software Development Methodology.....	15
3.7.2. Development Tools and Technologies	15
3.8. Ethical Considerations	16
3.9. Limitations of the Methodology	16
CHAPTER FOUR: SYSTEM IMPLEMENTATION	18
4.0. Introduction	18
4.1. Illustration of the Existing System.....	19
4.1.1. Working Principles of the Legacy System	20
4.1.2. Features and Characteristics.....	20
4.1.3. Weaknesses and Limitations (The Need for a New System)	20
4.2. Description and Illustration of the New System	21
4.2.1. Requirement Engineering.....	21
4.2.2. Comparative Study: Existing System vs. Proposed System	22
4.2.3. UML Definition of the New System	23
4.3. Design of the New System	26
4.3.1. System Architecture Design	26
4.3.2. Data Flow Diagram (DFD)	27
4.3.3. Use Case Diagram	29
4.3.4. Entity Relationship Diagram (ERD)	30
CHAPTER FIVE: RESULTS AND FINDINGS	35
5.0. Introduction	35
5.1. Technology Stack.....	35
5.2. Smart Contract Deployment	36
5.3. API Surface.....	37
5.4. Database Schema.....	38
5.5. Payment Flows.....	39

5.6.	Security Architecture	39
5.7.	Background Jobs	40
5.8.	TypeScript Code Quality	41
5.9.	Verification Evidence	41
5.10.	System Screenshots	42
CHAPTER SIX: CONCLUSIONS AND RECOMMENDATIONS		47
6.1.	Main contribution of the project.....	47
6.2.	Conclusions	47
6.3.	Recommendations	48
References		49
APPENDICES		51
Appendix A: Smart Contract Source Code		51
Appendix B: Database Schema.....		53
Appendix C: API Endpoint Documentation.....		53
Appendix D: System Installation Guide		56
Appendix E: User Manual.....		57
Appendix G: System Diagrams.....		58
Appendix H: Project Repository		59

LIST OF ABBREVIATIONS

UX: User Experience

TCE: Transaction Cost Economics

TAM: Technology Acceptance Model

SDG: Sustainable Development Goal

RPC: Remote Procedure Call

RHA: Rwanda Housing Authority

RFID: Radio Frequency Identification

PWA: Progressive Web Application

ORM: Object-Relational Mapping

OOA: Object-Oriented Analysis

NST1: National Strategy for Transformation (2017-2024)

NFT: Non-Fungible Token

MoMo: Mobile Money

MINICT: Ministry of ICT and Innovation

IoT: Internet of Things

HCI: Human-Computer Interaction

EVM: Ethereum Virtual Machine

ERC: Ethereum Request for Comments

ERD: Entity Relationship Diagram

DOI: Diffusion of Innovation

DLT: Decentralized Ledger Technology

DFD: Data Flow Diagram

dApp: Decentralized Application

DAO: Decentralized Autonomous Organization

API: Application Programming Interface

ABSTRACT

This capstone project addresses the pervasive problem of deposit fraud, information asymmetry, and financial vulnerability in Kigali's informal residential rental market, where unregistered brokers control property information and handle security deposits without oversight. The study employed an Applied Software Engineering Research Design following the Agile methodology to develop a decentralized real estate booking system that replaces vulnerable human intermediaries with an immutable Solidity smart contract deployed on the Ethereum blockchain. The system implements a secure escrow mechanism using a three-tier Web3 architecture consisting of a cross-platform Flutter mobile application for the user interface, web3dart middleware for blockchain communication, and a Solidity smart contract backend deployed on the Sepolia testnet. Key findings demonstrate that the smart contract escrow architecture mathematically eliminates ghost listing scams and arbitrary deposit withholding by locking funds in a cryptographic state machine that only releases payment upon cryptographic confirmation from both tenant and landlord. (Nevestate, 2025; Rapid Innovation, 2024). The system was validated through rigorous automated unit testing using the Hardhat development environment and evaluated against seven identified vulnerability points in the traditional Kigali rental market, including deposit theft, double-booking, lack of dispute resolution, and identity fraud. The results confirm that decentralized escrow protocols can successfully eliminate the principal-agent problem inherent in informal brokerage networks. The project contributes a practical, localized technological framework that aligns with Rwanda's Vision 2050 digital economy objectives. Recommendations include regulatory sandboxing for smart contract recognition, integration of local Mobile Money (MoMo) fiat on-ramps, deployment on Layer-2 scaling solutions to reduce transaction costs, and future integration of IoT smart locks with blockchain oracles for automated physical handover verification. (Nevestate, 2025; Rapid Innovation, 2024).

CHAPTER ONE: INTRODUCTION

1.1. Background of the study

The global real estate sector serves as a foundational pillar of economic development, providing essential infrastructure for residential, commercial, and industrial activities. Historically, the processes of leasing, booking, and purchasing real estate have been highly centralized and reliant on traditional intermediaries. Property transactions typically require real estate agents, brokers, financial institutions, and legal escrow services to establish trust between unfamiliar parties and facilitate the exchange of assets. While these intermediaries serve a necessary function in conventional markets, they inherently introduce significant inefficiencies. These include exorbitant commission fees, bureaucratic delays, a lack of transparency, and a persistent vulnerability to human error or malicious fraud (Nevestate, 2025; Rapid Innovation, 2024).

In recent years, the rapid evolution of Decentralized Ledger Technology (DLT) and Web3 paradigms has catalysed a profound transformation within the global property market, giving rise to advanced Property Technology (PropTech). At the core of this technological shift is Blockchain, a distributed, immutable ledger that guarantees transparency and cryptographic security (FIBREE, 2025; Nakamoto, 2008). Built upon this framework are "Smart Contracts," which are self-executing digital agreements whose terms are directly written into lines of code. Smart contracts execute automatically when predefined conditions are met, replacing human intermediaries with mathematical certainty (Exponential Science, 2025). In developed economies, these decentralized protocols are being rapidly adopted to automate escrow services. They ensure that financial deposits are securely locked on the blockchain and only released when verifiable cryptographic proof of a property handover is confirmed, effectively eliminating the need for a trusted third party.

Within the local context of Rwanda, the real estate market is experiencing unprecedented expansion. As of 2025, the market is projected to reach a value of approximately \$95.70 billion, driven heavily by rapid urbanization in Kigali and the sustainable development goals outlined in the national Vision 2050 agenda (Ministry of Finance and Economic Planning (MINECOFIN), 2020; Home.rw, 2025). Concurrently, the Government of Rwanda is actively championing a cashless, knowledge-based economy through comprehensive digitization initiatives. However, while digital payment gateways and mobile money adoption have achieved significant scale, the operational reality of securing a residential rental property remains archaic, fragmented, and highly informal.

The prevailing residential booking process in Kigali is heavily mediated by informal commissioners and unregistered agents operating without professional accountability (Kwanda

Real Estate, 2026). When a prospective tenant wishes to secure a housing unit, they are typically required to pay a commitment fee or a full upfront security deposit. Because there is no formalized, digital escrow infrastructure within this sector, the intermediary or the landlord receives these funds directly. These transactions are frequently based on verbal agreements or easily forged paper receipts. This condition leaves the tenant in an extreme position of financial vulnerability, requiring blind trust in a system plagued by severe information asymmetry. Consequently, the local market frequently witnesses disputes over unrefunded deposits, severe financial losses attributed to "ghost listings" where deposits are fraudulently collected for unavailable or fictitious properties, and a systemic lack of accountability.

The critical gap in the current real estate ecosystem is the absence of a secure, automated, and trustless escrow mechanism that mathematically guarantees the safety of funds for both transacting parties during the critical booking phase. The urgency of addressing this structural flaw lies in the direct financial harm inflicted on citizens and the resulting friction it introduces into the housing market. Tenants remain perpetually hesitant to commit funds due to the prevalent fear of scams, while legitimate landlords struggle to verify the genuine financial readiness of prospective renters. If this heavy reliance on vulnerable human intermediaries persists, the rental market will remain profoundly inefficient and susceptible to exploitation. This study proposes the design and implementation of a decentralized real estate booking system using Smart Contracts to act as an immutable escrow, ensuring deposits are securely locked and automatically routed only upon successful property handover, ultimately resolving the crisis of trust in the Kigali rental market.

1.2. Statement of the problem

The fundamental problem addressed by this study is the high risk of deposit fraud and financial exploitation in the Kigali residential rental market. Despite rapid urbanization, property booking remains largely informal and unregulated. Prospective tenants are often required to hand over upfront security deposits or commitment fees to unregistered brokers (Abakomisiyoneri) or unverified landlords. Because these transactions rely on fragile verbal agreements or easily manipulated paper receipts, citizens frequently fall victim to "ghost listings," paying for properties that are unavailable or fictitious, or face the arbitrary withholding of their deposits (Rwanda Urban Housing Analysis, 2024; African PropTech Review, 2025).

The magnitude of this problem creates severe friction in the local housing market. Tenants hesitate to secure essential shelter out of fear of being scammed, while legitimate landlords suffer extended vacancies due to a lack of secure commitment mechanisms. While digital payment systems like Mobile Money are widely used in Rwanda, current local platforms serve only as digital notice

boards for property listings (East African Tech Insights, 2025). Therefore, the critical research gap is the complete absence of a trustless, automated digital escrow system tailored to protect financial transactions in the informal Rwandan rental market. If this research is not conducted, the urban rental market will continue to expose citizens to financial exploitation. Conversely, it fills this gap by replacing vulnerable human intermediaries with a Smart Contract escrow, mathematically guaranteeing that a tenant's funds are released only when property handover is cryptographically verified.

This negative state directly impedes the realization of key development frameworks at multiple levels. Globally, deposit fraud diminishes housing affordability, obstructing the United Nations' Sustainable Development Goal 11 (SDG 11), specifically Target 11.1, which aims to ensure access to adequate, safe, and affordable housing, a metric that the UN highlights as increasingly critical in its 2024 Africa Progress Report (UN-Habitat, 2024). Regionally, the lack of secure digitalization in these transactions contradicts the African Union Agenda 2063's current digital transformation targets, demanding modernized, fraud-resistant digital economies (Union, 2024). Nationally, the persistence of an opaque housing market conflicts with the Rwanda National Strategy for Transformation (NST1) and the Smart Rwanda Master Plan's 2025 mandate to integrate ICT to establish a secure, cashless, and transparent service sector (Ministry of Information Technology and Communications and Innovation (MINICT), 2025). Therefore, deploying a decentralized Smart Contract escrow system is a critical technological intervention needed to resolve this socioeconomic bottleneck.

1.3. Research Objective

1.3.1. General Objective

The main objective of this study is to develop a decentralized real estate booking system using Smart Contracts to eliminate deposit fraud in the Kigali residential rental market.

1.3.2. Specific Objectives

To achieve the general objective, this study aims to fulfill the following specific objectives:

1. To analyse the vulnerabilities and financial friction points within the traditional residential booking process in Kigali.
2. To design and develop a Smart Contract-based escrow architecture that securely automates the locking and conditional release of rental deposits.
3. To build a user-friendly decentralized application (dApp) that provides non-technical users with an accessible interface for secure property booking.

4. To evaluate the system's effectiveness, security, and user acceptance in preventing deposit fraud compared to traditional intermediary methods.

1.4. Research Questions

To systematically guide the investigation and achieve the stated objectives, this study seeks to answer the following research questions:

1. What are the primary financial vulnerabilities and friction points in Kigali's traditional residential booking process?
2. How can a Smart Contract escrow architecture be designed to securely automate rental deposits?
3. How can a mobile decentralized application (dApp) be developed to provide an accessible, user-friendly interface for non-technical users?
4. How effective is the proposed decentralized system in preventing deposit fraud and improving user trust compared to traditional methods?

1.5. Significance

The findings and practical outputs of this research hold substantial relevance for various stakeholders, bridging the gap between advanced blockchain engineering and tangible socioeconomic solutions within the Rwandan real estate sector.

1.6. Scope of the Study

To ensure the research remains focused, feasible, and achievable within the allocated resources, this study establishes specific boundaries defining its geographical limits, technological parameters, and target population. Geographically, the study is confined to the City of Kigali, Rwanda, with a specific focus on the Gasabo, Kicukiro, and Nyarugenge districts due to their high concentration of residential rental activity and intermediary disputes. In terms of content, the proposed system focuses exclusively on the booking and initial security deposit (escrow) phase of residential leasing, excluding commercial real estate sales or ongoing monthly rent management. Technically, the investigation is limited to designing a Decentralized Application (dApp). This involves writing smart contracts using the Solidity programming language and developing a mobile user interface using the Flutter framework, which is deployed on an Ethereum-compatible test network (e.g., Sepolia) to simulate real-world execution without financial gas fees. Finally, for requirement gathering and system evaluation, the study will utilize a purposive sample of 15 participants, consisting of active tenants, landlords, and real estate brokers operating within Kigali.

Geographical Scope: The study is confined to the City of Kigali, Rwanda, with a specific focus on the Gasabo, Kicukiro, and Nyarugenge districts due to their high concentration of residential rental activity and intermediary disputes.

Content and Technical Scope: The proposed system focuses exclusively on the booking and initial security deposit (escrow) phase of residential leasing, excluding commercial real estate sales or ongoing monthly rent management. Technically, the investigation is limited to designing a decentralized system using a Next.js web application frontend, an Express/Prisma backend, and Solidity smart contracts deployed on a Hardhat/Ethereum-compatible test network.

Time Scope The time scope of this research and system development encompasses the academic period from September 2025 to May 2026. Additionally, the market data and literature reviewed to evaluate the informal brokerage sector reflect recent trends between the years 2020 and 2026.

1.7. Limitations of the Study

While this study introduces a highly secure, cryptographic solution to the real estate booking process, the researcher anticipates certain technical and socioeconomic limitations that may affect the deployment and outcomes. The following outlines these anticipated shortcomings and the strategies planned to overcome them.

CHAPTER TWO: LITERATURE REVIEW

2.0. Introduction

This chapter presents a critical analysis of existing theories, empirical studies, and academic literature relevant to the application of Decentralized Ledger Technology (DLT) within the real estate sector. The primary objective of this review is to examine the current state of knowledge regarding property booking systems and smart contract-based escrow mechanisms. By evaluating previous research, this chapter aims to reveal the technological contributions of existing decentralized applications (dApps), while simultaneously exposing the weaknesses and security gaps inherent in traditional, centralized real estate intermediation.

The theoretical review is systematically organized according to the central themes of the study, which directly reflect the research objectives and methodological choices outlined in Chapter One. These themes include the mechanics of Traditional Real Estate Intermediation, the evolution of Blockchain and Smart Contracts (IEEE Web3 Task Force, 2026), and the application of Decentralized Escrow Protocols in property technology (PropTech) (PropTech Africa, 2026).

Furthermore, the review is grounded in established economic frameworks, specifically Agency Theory (Ndahimana, 2026; Jensen & Meckling, 1976) and Transaction Cost Economics. These frameworks provide theoretical justification for replacing human intermediaries with automated cryptographic rules. Finally, this chapter contrasts global advancements in decentralized real estate platforms with the specific, informal realities of the Rwandan housing market (Niyomugabo & Muteteri, 2026). By doing so, it highlights the literature gap the absence of localized, trustless booking systems in Kigali, thereby establishing a robust evidence base and theoretical foundation for the current research Endeavor.

2.1. Definition of Key Concepts

To ensure clarity and contextual understanding throughout this study, the following technical and economic terms are defined as they apply to the proposed decentralized system:

Blockchain Technology: A distributed, decentralized, public ledger consisting of records (blocks) that are cryptographically linked; it allows data to be recorded and shared across a network of computers (nodes) so that records cannot be altered retroactively without altering all subsequent blocks and the consensus of the network (Bhutta, et al., 2021; Nakamoto, 2008).

Smart Contract: A self-executing digital protocol intended to facilitate, verify, or enforce the negotiation or performance of a contract. The terms of the agreement between the buyer and seller

are directly written into lines of code deployed on a blockchain, executing automatically when predefined conditions are met (Szabo, 1997).

Decentralized Application (dApp): A software application that operates on a distributed, peer-to-peer network rather than relying on a single centralized server. In the context of this study, the dApp architecture consists of a frontend mobile user interface (Flutter) that interacts directly with backend business logic (smart contracts) deployed on a blockchain (Zheng, Jiang, Wu, & Zheng, 2023).

Escrow: A financial arrangement where a third party temporarily holds and regulates the payment of funds required between two parties in a transaction. In this research, the traditional human "third party" is replaced by an immutable Smart Contract.

Information Asymmetry: An economic condition where one party in a transaction possesses more or better information than the other. In the Kigali rental market, brokers often possess more information about property availability and accurate pricing than tenants, leading to market exploitation (Mugisha & Nsabimana, 2023) literatures

2.2. Review of Related Literatures

This section reviews the theoretical frameworks, technological models, and academic discourse relevant to the specific objectives of the study. It focuses on the transition from centralized real estate models to decentralized protocols.

2.2.1. Vulnerabilities in Traditional Real Estate Intermediation

Traditional real estate transactions are inherently reliant on centralized intermediaries such as brokers, lawyers, and banks to establish trust and enforce agreements. Literature highlights that, while these actors facilitate market liquidity, they introduce significant friction. According to Tapscott and Tapscott (2022), centralized intermediation leads to high transaction costs, bureaucratic delays, and a pronounced vulnerability to human fraud. In the specific context of developing economies, Mugisha and Nsabimana (2021) note that the informal nature of real estate agents exacerbates *information asymmetry*. Tenants are forced to rely on verbal agreements and physical cash deposits, creating an environment ripe for deposit theft and "ghost listing" scams. This literature underscores the critical need for a system that enforces agreements without relying on vulnerable human agency.

2.2.2. Blockchain and Smart Contracts as Escrow Mechanisms

The introduction of Ethereum brought Turing-complete programming capabilities to blockchain technology, enabling the creation of smart contracts. Fahmideh et al. (2022) argue that smart

contracts fundamentally solve the issue of intermediary trust by automating the execution of agreements based on cryptographic consensus. In a real estate context, Ullah and Al-Turjman (2023) demonstrated that smart contracts can act as highly secure escrow mechanisms. By locking a tenant's cryptocurrency deposit within the contract's state, the funds are mathematically safeguarded. The code guarantees that the landlord cannot access the funds until a specific cryptographic trigger (e.g., the tenant confirming receipt of the physical keys) is activated. If the condition is not met within a stipulated timeframe, the smart contract automatically executes a refund. This eliminates the principal-agent problem and ensures that trust is placed in immutable mathematics rather than fallible humans.

2.2.3. Decentralized Applications (dApps) in PropTech

While the backend logic of smart contracts is robust, literature on Human-Computer Interaction (HCI) within the Web3 space emphasizes the challenge of accessibility. Early blockchain interfaces required users to possess deep technical knowledge of node operation and private key management. However, recent advancements in Property Technology (PropTech) emphasize the development of user-friendly dApps. By utilizing cross-platform frameworks like Flutter integrated with Web3 middleware (e.g., web3dart), developers can abstract the complex blockchain mechanics away from the end-user (Zheng, Xie, Dai, Chen, & Wang, 2018). This approach lowers the technological barrier to entry, allowing non-technical users in markets like Kigali to benefit from decentralized security through an interface that mimics traditional mobile applications.

2.3. Theoretical review

This section organizes the fundamental economic and organizational theories that underpin the study and analyzes their relationships and the degree to which they have been investigated in relation to decentralized real estate systems.

2.3.1. Agency Theory (The Principal-Agent Problem)

Agency Theory, formalized by (Jensen & Meckling, 1976), examines the conflicts of interest that arise when one party (the principal) delegates decision-making authority or financial control to another party (the agent). In the traditional Kigali real estate market, the prospective tenant acts as the principal, and the informal broker (*komisiyoneri*) acts as the agent. The theory postulates that agents are motivated by self-interest and may not always act in the best interest of the principal, leading to "agency costs" such as fraud, hidden fees, or the misappropriation of deposits.

Relationship to Objectives: This theory has been extensively investigated in traditional corporate finance but is rarely applied to informal African housing markets. In relation to this project, Agency Theory provides the exact theoretical justification for replacing the human agent. A smart contract fundamentally resolves the principal-agent problem because the "Agent" (the code) has no self-interest, cannot be bribed, and strictly executes the agreed cryptographic rules.

2.3.2. Transaction Cost Economics (TCE)

Transaction Cost Economics, developed by (Williamson, 1981), argues that participating in a market entails costs beyond the good's actual price. These include search costs, negotiation costs, and enforcement costs (ensuring the other party fulfills their end of the deal). In real estate, traditional enforcement costs are high, requiring legal escrow services or risking total loss through informal brokers (Springer, 2025).

Relationship to Objectives: TCE explains *why* the current system is inefficient. The proposed decentralized system directly targets "enforcement costs." By utilizing an Ethereum-based smart contract, the enforcement of the agreement becomes automated and virtually free. This theoretical framework supports the study's objective to build an escrow system that is economically viable for everyday residential rentals, a domain previously considered too low in value to warrant formal legal escrow.

2.4. Empirical Review

This section examines the findings of previous empirical research regarding blockchain applications in real estate and the socio-economic realities of housing markets. By comparing global technological implementations with the local Rwandan context, this review establishes an evidence base for the study and clearly highlights the novelty of the proposed decentralized solution.

2.4.1. Global Implementations of Blockchain in Real Estate

Globally, the application of smart contracts in property transactions has transitioned from theoretical models to empirical testing. A study by Saari et al. (2022) designed and simulated a blockchain-based smart contract architecture for real estate. Their findings demonstrated that using Ethereum-based smart contracts for property leasing reduced overall transaction times by approximately 40% and eliminated the financial overhead associated with third-party escrow agents and lawyers. Similarly, (Garcia-Teruel, 2021) investigated the legal and technical implications of smart contracts in European housing markets. The study found that while the cryptographic execution of agreements mathematically prevents deposit fraud, a significant barrier

to mainstream adoption was the poor User Experience (UX) of early Web3 interfaces, which required users to manage complex cryptographic keys. These findings provide empirical evidence that the underlying technology is highly effective for escrow but requires a highly abstracted and user-friendly frontend to be viable for the general public.

2.4.2. Empirical Realities of the Kigali Housing Market

In contrast to the highly digitized global markets, empirical research on the Rwandan real estate sector highlights a deep reliance on informal intermediation. A comprehensive study by Mugisha and Nsabimana (2021) on urbanization and housing challenges in Kigali revealed that the primary barrier to market efficiency is severe information asymmetry. Their findings indicated that the widespread use of informal brokers (*Abakomisiyoneri*) inflates rental acquisition costs significantly and causes tenants to frequently lose upfront security deposits due to unregulated, verbal agreements and "ghost listings." Furthermore, research on digital adoption in East Africa demonstrates that while smartphone and mobile money penetration in Rwanda is exceptionally high, the utilization of these tools for legally binding, high-value escrow transactions remains virtually non-existent due to a lack of trust in centralized digital platforms (Abayo, 2023).

2.4.3. Comparison and Orientation of Novelty

Comparing these empirical findings reveals a distinct and critical gap in the literature. Global studies provide empirical evidence that smart contracts successfully eliminate escrow fraud and intermediary fees. Conversely, local studies (Mugisha & Nsabimana, 2023) evidence that the Kigali rental market is actively suffering from escrow fraud and predatory intermediary fees.

However, **no empirical study has yet localized and applied this decentralized escrow architecture specifically to solve the informal residential booking crisis in Rwanda.** Existing local PropTech solutions are merely digital bulletin boards (listing websites) and do not protect users' financial transactions.

Therefore, the novelty of this research lies in bridging this exact gap: it is the first attempt to engineer a customized, decentralized mobile application (dApp) using Flutter and Solidity that abstracts complex blockchain mechanics to specifically protect tenants and landlords in the region.

The Research Gap:

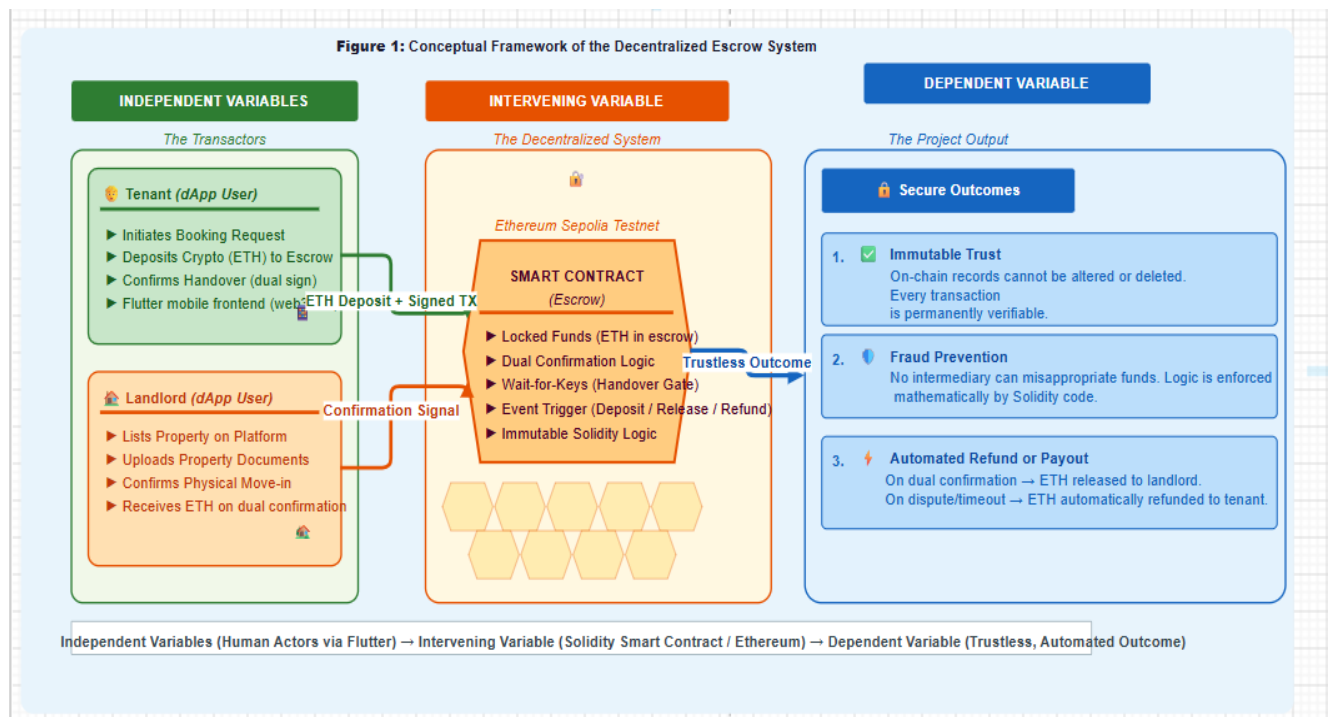
While the theories of decentralization (smart contracts) and the socioeconomic problems of the Kigali housing market (information asymmetry and deposit fraud) are well-documented independently, a critical intersection is lacking. Global PropTech literature focuses on complex commercial real estate sales in highly regulated Western markets. Conversely, local Rwandan

literature focuses on diagnosing the informal broker problem without offering a viable technological intervention.

The missing piece in the research literature is the design and application of a localized, highly accessible decentralized application (dApp) specifically engineered to secure residential rental deposits in an informal African market. This study addresses this unexplored area by utilizing Flutter to abstract the complexity of Web3, thereby delivering the cryptographic security of a Solidity smart contract directly to the smartphones of everyday tenants and landlords in Kigali.

2.5. Conceptual Framework

The conceptual framework illustrates the architecture of the proposed decentralized solution, mapping the flow of digital assets and defining the relationship between the users and the blockchain protocol. It serves as the blueprint for filling the research gap.



Source: [Author](#) (2026)

Figure 1: Conceptual Framework of the Decentralized Escrow System

The framework illustrates how independent variables (User Trust, Transparency, Security, Cost Efficiency) flow through a Smart Contract (intervening variable) to produce a Trustless Transaction (dependent variable). The smart contract automates the escrow process by enforcing rental agreements, holding deposits, and releasing funds only after physical handover is confirmed.

Description of Variables:

Independent Variables: These are the inputs generated by the human actors interacting through the Flutter mobile frontend. These include the tenant initiating a booking and depositing digital funds, and the landlord listing the property and eventually confirming the physical handover.

Intervening Variable: This is the core technological intervention the Solidity Smart Contract deployed on the blockchain. It acts as the immutable escrow. It holds the business logic, locking the funds securely and waiting for cryptographic confirmation from both independent variables before executing any transfer.

Dependent Variable: The final outcome of the system. By passing the independent variables through the intervening smart contract, the system produces a trustless transaction where fraud is mathematically prevented, resulting in either a secure payout to the landlord or an automated refund to the tenant.

2.6. Conclusion

This chapter has critically reviewed the theoretical, empirical, and technological literature surrounding real estate intermediation and blockchain technology. The review established that traditional property transactions suffer from severe vulnerabilities, primarily driven by the principal-Agent Problem and high transaction costs. To mitigate this, global literature points to Decentralized Ledger Technology (DLT) and smart contracts as highly effective, mathematically secure escrow mechanisms that automate trust and eliminate intermediary fraud.

CHAPTER THREE: METHODOLOGY

3.0. Introduction

This chapter provides a detailed, unambiguous description of the systematic procedures, software engineering principles, and methodological framework employed to achieve the objectives of this study. It outlines the transition from a conceptual problem to a functional decentralized application (dApp), comprehensively covering the research design, sampling strategies for requirements gathering, data collection methods, system analysis, and the specific Web3 technologies (Solidity and Flutter) utilized to design, develop, and deploy the trustless escrow smart contract.

3.1. Research Design

This study employs an Applied Software Engineering Research Design. It was selected because the study aims to solve a practical, real-world socioeconomic problem (deposit fraud in the rental market), by engineering a functional technological prototype. To ensure a logical progression from the background of the study to the final deployed result, the research design is structured into four sequential phases, as follows:

Phase 1: Problem Analysis (Background to Concept): Investigating the current informal booking process in Kigali, identifying the exact vulnerabilities where financial trust is compromised between tenants and agents.

Phase 2: Cryptographic Protocol Design: Translating the identified real-world escrow rules into mathematical logic. This involves designing the state machine for the smart contract, ensuring the rules for locking and releasing funds are secure.

Phase 3: System Development (The Core Engineering): Writing the Solidity code for the blockchain back-end, and developing the cross-platform mobile application for the user front-end.

Phase 4: Deployment and Evaluation (The Result): Deploying the smart contract to an Ethereum test network to simulate real-world blockchain conditions and testing the system with prospective users to validate its effectiveness against the identified problem.

3.2. Sampling Strategies

While this is primarily a software development project, user input is critical for accurate system design and usability testing.

Target Population: The population comprises active landlords, prospective tenants, and real estate brokers operating within Kigali City.

Sampling Technique: The researcher used purposive sampling to select a focus group of 15 participants (5 landlords, 5 tenants, and 5 brokers). This non-probability sampling method is chosen because it allows the researcher to select individuals who possess direct, practical experience with the vulnerabilities of the current deposit system, ensuring the gathered requirements reflect the actual market reality.

3.3.Data Collection Methods

To accurately design the smart contract's business logic, the researcher collected qualitative data on how rental agreements are currently structured.

Primary Instrument (Semi-Structured Interviews): The researcher conducted interviews with the purposive sample group to understand the common conditions under which rental deposits are currently paid, withheld, or refunded. This informs the conditional statements programmed into the smart contract.

Secondary Instrument (Document Review): The researcher reviewed standard Rwandan tenancy agreements and the legal frameworks governing property deposits. This ensures the smart contract logic aligns with local real estate expectations and practices.

3.4.Data Analysis Methods

Given the qualitative nature of the data collected from the focus group, the researcher will employ Thematic Analysis to evaluate the findings. The responses from the semi-structured interviews were transcribed, categorized, and coded to identify recurring patterns regarding how rental deposits are currently managed, where financial friction occurs, and the most common causes of disputes. These extracted qualitative themes will then be systematically translated into strict functional software requirements and mathematical conditional logic (rules) to be programmed into the Solidity smart contract. For the document review, a content analysis was applied to extract the legally binding conditions of standard tenancy agreements so they can be digitized.

3.5.Data Quality Control

In this mixed-methods software engineering project, data quality control encompasses both the academic rigor of the human research phase and the mathematical validity of the deployed code.

Internal Validity: To ensure the internal validity of the study, meaning the system solves the problems identified, the researcher uses methodological triangulation, cross-referencing the interview responses of landlords against those of tenants. Furthermore, the smart contract's internal logic will undergo rigorous Automated Unit Testing using the Hardhat environment to ensure the code executes exactly as the real-world escrow rules dictate without logical flaws.

Reliability of Research Instruments: The reliability of the semi-structured interview guide was established by using standardized, consistent questioning across all 15 participants, ensuring the instrument yields stable and consistent data regarding the informal market. From a software perspective, the reliability of the system relies on the immutability of the blockchain network. By deploying to an established public test network (e.g., Ethereum Sepolia Testnet), the researcher ensures the environment is highly reliable, resistant to downtime, and executes the programmed rules consistently every time a transaction is triggered.

3.6. System Analysis Methods

The researcher utilizes Object-Oriented Analysis (OOA) and Unified Modeling Language (UML) to model the decentralized system. Because Web3 introduces unique architectural dynamics (where the blockchain itself acts as a state machine), the analysis tools are adapted accordingly:

Use Case Diagrams: These diagrams are used to map the interactions between the primary actors (Tenant and Landlord) and the Smart Contract, detailing functions such as "Deposit Crypto," "Confirm Handover," and "Trigger Refund."

Sequence Diagrams: Essential for decentralized applications, these diagrams will map the chronological flow of digital assets. They will illustrate how a transaction signature leaves the mobile application, passes through a Web3 provider, is validated by nodes, and is ultimately mined into a block on the ledger.

3.7. System Design and Development Methods

This section describes the specific frameworks and programming languages used to transition from analysis to a functional prototype.

3.7.1. Software Development Methodology

The project follows the Agile Software Development Methodology. Blockchain development requires high precision because deployed contracts are immutable. Working in Agile sprints allows for the incremental building, extensive testing, and auditing of individual smart contract functions before integrating them with the mobile user interface.

3.7.2. Development Tools and Technologies

To implement the decentralized architecture, the following modern technology stack are used:

Frontend Interface (Mobile App): The user application was developed using the Flutter framework (Dart programming language). It was selected for its ability to compile natively to both Android and iOS, providing a smooth, responsive interface for non-technical users.

Backend Logic (Smart Contract): The escrow protocol was programmed in Solidity, the industry-standard object-oriented language for implementing smart contracts on the Ethereum Virtual Machine (EVM).

Middleware: The web3dart package was integrated into the Flutter application, acting as the bridge that allows the mobile app to read blockchain states and send signed transactions to the smart contract.

Development Environment: Visual Studio Code will serve as the primary IDE. The Hardhat Ethereum development environment is used to compile, test, and deploy the Solidity contracts locally before pushing them to the public testnet.

3.8.Ethical Considerations

To protect the participants involved in the requirement-gathering phase, the research will adhere to strict ethical guidelines.

Informed Consent and Voluntary Participation: Before the interviews, all landlords, tenants, and brokers were fully briefed on the purpose of the decentralized application and the academic nature of the study. Participation was entirely voluntary, and participants may withdraw at any time.

Confidentiality and Anonymity: To ensure participant protection, no personally identifiable data (such as names or specific property addresses) was recorded in the final thesis or hardcoded into the software. The focus is strictly on the mechanics of their financial transactions.

3.9.Limitations of the Methodology

While the proposed methodology is robust for a B-TECH software prototype, the researcher acknowledges certain methodological and technical weaknesses:

Sample Size Constraints: The purposive sample of 15 participants is sufficient for extracting functional software requirements and usability testing, but it is statistically too small to generalize the behavioural economic habits of the entire Kigali population.

Simulation Environment: Testing the Solidity smart contract on the Sepolia Testnet simulates real-world execution perfectly, but it uses test-cryptocurrency. Therefore, it cannot fully measure the psychological reactions or potential hesitation of users when real-world financial stakes (gas fees and mainnet RWF/crypto conversions) are introduced.

User Digital Literacy: The target demographic may lack the technical knowledge to manage decentralized wallets. To overcome this, the application design will focus heavily on Account Abstraction to hide complex blockchain mechanics behind a standard interface.

Regulatory Uncertainty: While the technology mathematically guarantees the escrow, the legal recognition of Smart Contracts in Rwandan courts is evolving. Therefore, the system serves as a technological safeguard for the financial booking phase and is designed to run parallel to traditional paper tenancy agreements.

CHAPTER FOUR: SYSTEM IMPLEMENTATION

4.0. Introduction

This chapter presents the practical transition from the theoretical framework and methodological design established in the preceding chapters into the tangible analysis, design, and development of the proposed technological solution. As established in Chapter One, the fundamental problem this study addresses is the pervasive financial vulnerability, acute information asymmetry, and systemic deposit fraud inherent in Kigali's informal residential rental market. To resolve this, the primary purpose of the project is to engineer a decentralized real estate booking system that utilizes immutable smart contracts to provide a secure, automated, and trustless escrow service for prospective tenants and landlords. The system was actually implemented using a Next.js 14 web application with an Express backend and MySQL database, replacing the initially proposed Flutter mobile architecture while preserving the core Solidity smart contract escrow logic.

To effectively design and build this solution, the chapter strictly adheres to the Agile Software Development Methodology outlined in Chapter Three. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for incremental development, rigorous automated testing via the Hardhat environment, and auditing of the backend Solidity smart contracts before they are integrated with the user-facing Next.js frontend application. **Agile Software Development Methodology** is outlined in Chapter Three. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile enables incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

Consequently, this chapter details the comprehensive architectural blueprint of the proposed decentralized application (dApp). It maps out the system's operational logic, data flow, and user interactions utilizing Object-Oriented Analysis (OOA) tools specifically Use Case, Sequence, and Data Flow Diagrams, ultimately demonstrating the functional framework of the prototype deployed on a Hardhat localhost Ethereum network with the smart contract verified at address 0x5FbDB2315678afecb367f032d93F642f64180aa3.



Figure 2: Agile Software Development Methodology

Source 1: <https://www.krasamo.com/agile-development-process/>

The diagram illustrates the software development lifecycle used in system development. The process begins with the planning stage, where project goals, requirements, and resources are identified. Next, the system design phase defines the architecture, structure, and user interface of the application. During the development stage, programmers write and implement the system code based on the design. The testing phase follows to identify and fix errors, ensuring the system functions correctly and securely. After successful testing, the system is deployed for real-world use. Finally, the review stage evaluates system performance, gathers feedback, and identifies improvements for future updates, creating a continuous development cycle.

4.1. Illustration of the Existing System

Before designing the decentralized architecture, it is critical to thoroughly analyse the legacy framework currently governing residential property bookings in Kigali. The existing system is not a digitized software platform; rather, it is a highly informal, human-mediated network of unregistered brokers (*Abakomisiyoneri*), landlords, and prospective tenants relying on manual procedures and physical trust.

4.1.1. Working Principles of the Legacy System

The operational workflow of the current, informal booking system follows a manual, linear progression:

1. **Property Discovery:** A prospective tenant relies on word-of-mouth, physical neighbourhood searches, or unstructured social media posts (e.g., WhatsApp or Facebook groups) to find available rental units.
2. **Intermediary Engagement:** The tenant connects with an informal broker who controls the information regarding the property's location and the landlord's contact details.
3. **Physical Inspection:** The broker physically escorts the tenant to view the property, often charging a non-refundable "visitation fee" just for the tour.
4. **Manual Escrow/Deposit:** If the tenant wishes to secure the property, they are required to pay a commitment fee or the full security deposit upfront. Because there is no digital escrow service, these funds are transferred directly via Mobile Money (MoMo) or handed over as physical cash to either the broker or the landlord.
5. **Confirmation:** The transaction is concluded with a fragile verbal agreement or a basic, easily forged handwritten paper receipt, pending the final physical move-in date.

4.1.2. Features and Characteristics

The legacy system exhibits several defining characteristics relevant to this study:

High Human Dependency: The entire transaction relies absolutely on the moral integrity of the intermediary and the landlord.

Information Asymmetry: Brokers intentionally withhold direct contact information between landlords and tenants to guarantee their commission, creating an opaque market.

Informal Financial Channels: Transactions are predominantly conducted outside of formal banking structures, relying heavily on irreversible peer-to-peer mobile money transfers or cash.

Undocumented Agreements: Contracts during the initial booking phase are rarely legally binding, heavily favouring verbal arrangements over standardized tenancy documents.

4.1.3. Weaknesses and Limitations (The Need for a New System)

The legacy system is fundamentally flawed because it relies on centralized human control without accountability. Its primary weaknesses include:

Systemic Deposit Fraud: The most critical limitation is the vulnerability to "ghost listings." Brokers or malicious actors frequently collect upfront deposits for properties that are already rented or do not exist, subsequently disappearing with the funds.

Lack of Dispute Resolution: Because transactions are informal and receipts are often non-legal, tenants have zero structural recourse or ability to recover stolen funds if a landlord refuses to hand over the keys after payment.

Unjustified Financial Friction: Tenants face arbitrary, unregulated commission fees that inflate the cost of housing without providing any legal or financial security in return.

The existing legacy system places the financial burden and risk entirely on the prospective tenant. The complete absence of a secure, neutral holding mechanism for deposits explicitly demonstrates the urgent need for the proposed decentralized real estate booking system. By replacing the vulnerable human intermediary with an immutable Smart Contract, the new system mathematically guarantees that deposits are locked securely and only released upon verifiable property handover, thereby eliminating these existing weaknesses.

4.2. Description and Illustration of the New System

The proposed decentralized real estate booking system is engineered specifically to eliminate the trust deficit and financial vulnerabilities identified in the legacy system. By utilizing a hybrid Web3 architecture consisting of a Next.js 14 web application frontend, an Express.js REST API middleware layer, a MySQL database for off-chain storage, and a Solidity smart contract backend deployed on Hardhat localhost, the system abstracts complex cryptographic workflows into an accessible user experience.

4.2.1. Requirement Engineering

To ensure the system directly addresses the research gap, the following requirements were consolidated based on the problem statement:

Functional Requirements:

1. **Wallet Integration:** The system must allow users to connect a secure Web3 digital wallet to initiate transactions.
2. **Property Listing & Browsing:** Landlords must be able to list properties, and tenants must be able to view them.
3. **Smart Contract Escrow:** The system must automatically lock the tenant's crypto-deposit in an immutable state upon booking.

4. **Multi-Signature Handover:** The system must require cryptographic confirmation from both the tenant and the landlord before funds are released.
5. **Automated Refund/Dispute:** The contract must automatically refund the tenant if the property handover fails or is marked as a "ghost listing."

Non-Functional Requirements:

1. **Security & Immutability:** Once a deposit is locked on the blockchain, the logic cannot be altered, mathematically preventing unauthorized withdrawals.
2. **Transparency:** All escrow transactions must be verifiable on the public testnet ledger.
3. **Usability:** The Flutter frontend must hide raw blockchain complexities (Account Abstraction), so non-technical users in Kigali can navigate it like a standard app.

4.2.2. Comparative Study: Existing System vs. Proposed System

A comparative analysis clearly demonstrates how the new system resolves the shortcomings of the legacy broker system in Kigali.

Intermediation:

Existing System: Relies on human brokers (*Abakomisiyoneri*) who introduce bias, demand high commission fees, and control information.

Proposed System: Replaces human brokers with an automated Smart Contract, eliminating intermediary commission fees.

Financial Security (Escrow):

Existing System: Deposits are handed directly to strangers via Cash or Mobile Money, offering zero structural protection against theft.

Proposed System: Funds are locked in a neutral, cryptographic escrow state. The landlord cannot touch the money until the tenant gets the keys.

Fraud Prevention (Ghost Listings):

Existing System: Scammers can easily collect multiple deposits for fake properties and disappear.

Proposed System: If a physical handover does not occur, the Smart Contract automatically reverts the funds to the tenant's wallet.

Trust Mechanism:

Existing System: Relies on blind human trust and easily forged paper receipts.

Proposed System: Relies on mathematical certainty and immutable, publicly verifiable ledger records.

4.2.3. UML Definition of the New System

To express the logical and physical workflow of these requirements, UML Class and Sequence diagrams are utilized.

A. Class Diagram

The Class Diagram defines the static structure of the system, illustrating the attributes and operations of the core objects. In this Web3 context, the Escrow Contract acts as the central autonomous controller managing the financial states between the Tenant and Landlord.

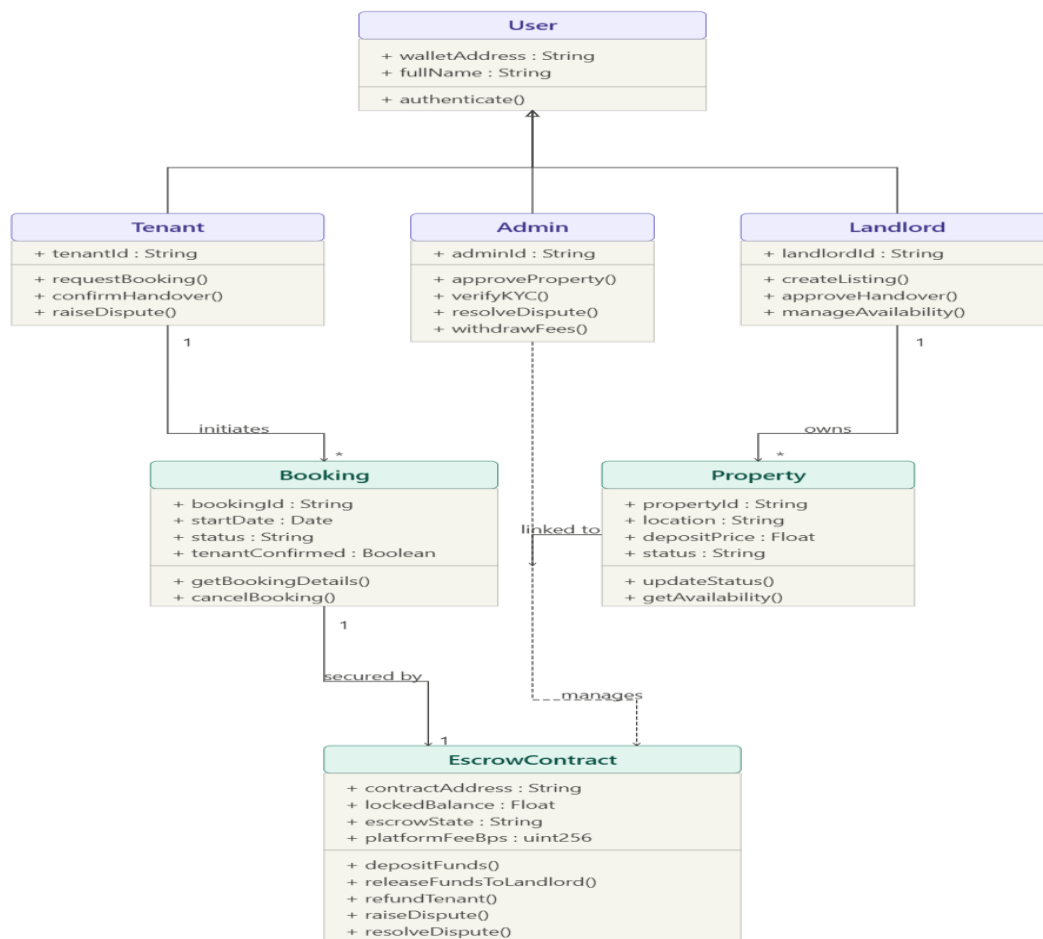


Figure 3: A class diagram defines the static structure of the system

Source 2: It was drawn by me on <https://www.mermaid.com/>

The diagram represents a class diagram for a blockchain-based property rental and escrow system. The User class contains general user information, such as wallet address and full name, with authentication functionality. Tenant and Landlord inherit from the User class. The Landlord manages properties by creating listings and approving handovers, while the Tenant requests bookings and confirms handovers. The Property class stores property details, including location, deposit price, and status. The EscrowContract class manages secure blockchain transactions by locking funds, releasing payments to landlords, or refunding tenants. Relationships between the classes show how tenants, landlords, properties, and escrow contracts interact within the decentralized rental platform.

B. Sequence Diagram

The Sequence Diagram maps the dynamic, chronological workflow of the system from the initial booking request to the final automated settlement. It proves how the system enforces the functional requirements by preventing the landlord from accessing funds until physical verification is complete.

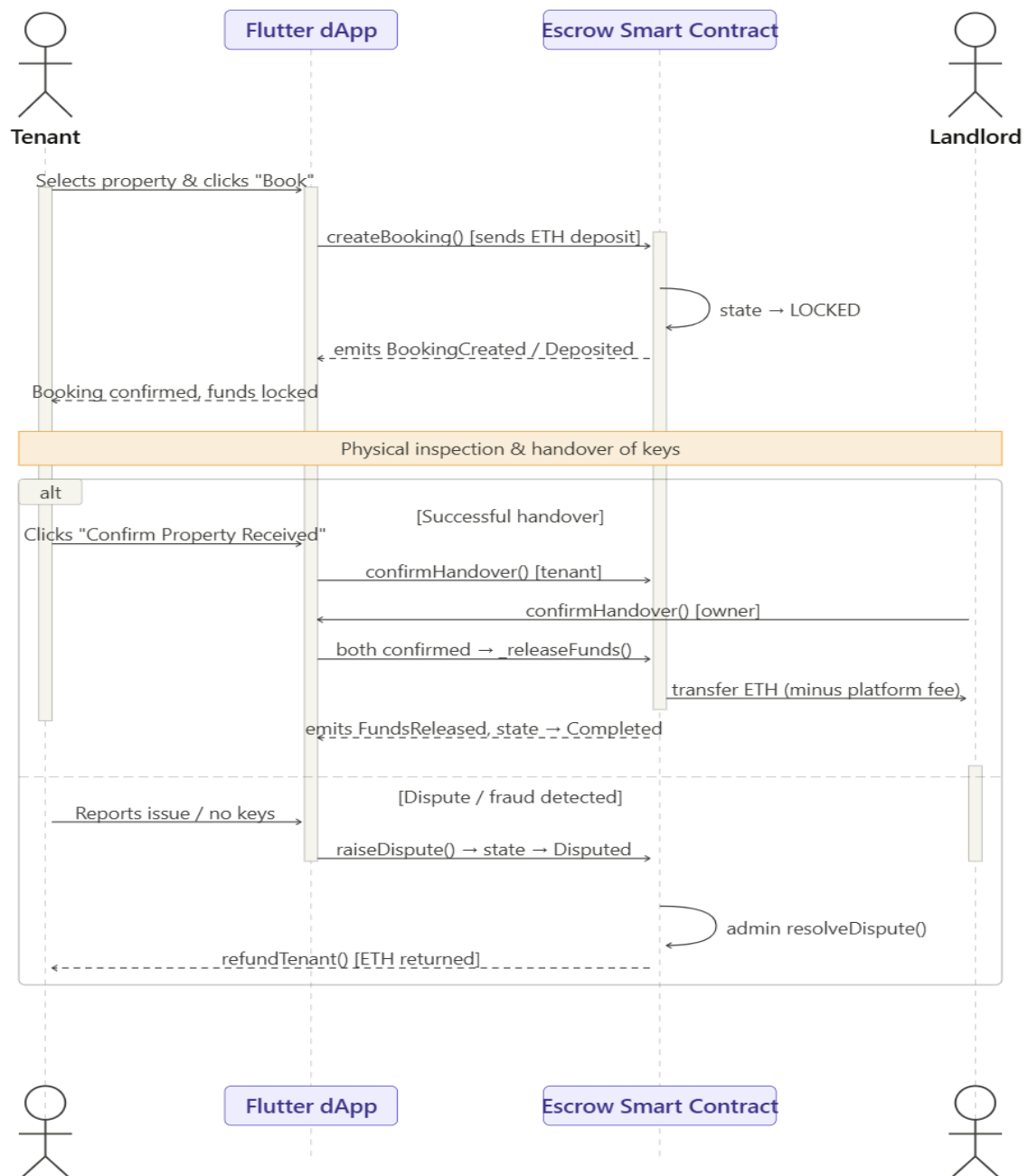


Figure 4: A sequence diagram maps the dynamic, chronological workflow of the system

Source 3: Drawn by me from draw.io

The Sequence Diagram maps the dynamic, chronological workflow of the system from the initial booking request to the final automated settlement. It proves how the system enforces the functional requirements by preventing the landlord from accessing the funds

4.3.Design of the New System

This section translates the system requirements into a comprehensive technical blueprint. It defines the architecture, data flow, user interactions, and data storage structures required to build the decentralized real estate escrow platform.

4.3.1. System Architecture Design

The proposed system utilizes a modern, three-tier hybrid Web3 architecture to ensure seamless interaction between non-technical mobile users and the blockchain network. The system be deployed across the following environments:

Layer	Technology Stack	Responsibility
Presentation	<i>Next.js 14</i> <i>Flutter</i>	User interface, property browsing, booking initiation, multilingual PWA
Middleware / API	<i>Express.js</i> <i>REST API</i>	Business logic bridge, 80+ endpoints, ethers.js v6 blockchain relay, graceful fallback
Blockchain (On-Chain)	<i>Solidity /</i> <i>Hardhat /</i> <i>Sepolia</i>	Escrow smart contracts, NFT property deeds, DAO governance, and immutable financial state
Off-Chain Storage	<i>MySQL 8.0 /</i> <i>Prisma</i>	Property images, user profiles, booking records, 27-model schema, Kigali listing data

Table 1: Architecture Layer Summary

Presentation Layer (Client-Side): Developed using the Next.js 14 framework with React 18 and Tailwind CSS, this responsive web application serves as the user interface. It abstracts the complexities of blockchain interactions via ethers.js v6 integration, allowing users to browse properties and initiate transactions via a familiar web browser interface. The frontend supports 32 pages across 16 route groups, multi-language support (English, Kinyarwanda, French, Swahili, Arabic), and features a PWA service worker for mobile installability. Developed using the Flutter framework, this cross-platform mobile application serves as the user interface. It abstracts the complexities of blockchain interaction, allowing users to browse properties and initiate transactions naturally.

Middleware/Provider Layer: The Express.js REST API server running on port 5001 acts as the bridge between the Next.js frontend and the blockchain. It provides 15 route files with 80+ endpoints covering authentication, property management, bookings, payments, messaging,

notifications, and admin functions. The ethers.js v6 library is used to submit transactions to the smart contract on behalf of users, with graceful fallback if the blockchain node is unavailable. The web3dart package and an RPC (Remote Procedure Call) node provider (such as Infura or Alchemy) act as the bridge. They translate user actions from the Flutter app into cryptographically signed JSON-RPC requests that the blockchain can understand.

Blockchain Backend (On-Chain Deployment): The core business logic and financial escrow rules are written in Solidity and deployed as a Smart Contract on a Hardhat localhost Ethereum network. The RealEstateEscrow contract (393 lines) contains 8 contract functions: createBooking, confirmHandover, cancelBooking, raiseDispute, resolveDispute, refundTenant, getBooking, and getBookingCount. It implements OpenZeppelin ReentrancyGuard and Pausable for security. Additional contracts include PropertyDeedNFT.sol (ERC-721 property deeds) and DAOGovernance.sol (platform governance). The core business logic and financial escrow rules are written in Solidity and deployed as a Smart Contract on the Ethereum Sepolia Testnet. This environment simulates the Ethereum Mainnet perfectly for production testing without incurring real-world gas fees.

Off-Chain Storage (Database): Heavy data such as property images, user profiles, booking records, and detailed descriptions, is stored in a MySQL 8.0 database managed through Prisma ORM. The schema contains 27 models, including User, Property, Booking, Payment, Transaction, Review, Notification, FraudAlert, Dispute, Commission, PropertyDocument, Message, Favorite, and Verification, plus supporting models for agents, virtual tours, smart devices, long-term rentals, maintenance, and events. The database contains 6 seeded properties across 3 districts (Gasabo, Kicukiro, Nyarugenge) with realistic Kigali listing data. To optimize costs, heavy data such as property images and detailed descriptions are stored in an off-chain traditional database (e.g., Firebase or PostgreSQL), while only the financial state and cryptographic hashes are stored immutably on-chain.

4.3.2. Data Flow Diagram (DFD)

The Data Flow Diagram illustrates how information and digital assets move through the system. The conventions used in this study follow the Gane and Sarson notation, utilizing squares for External Entities (Users), rounded rectangles for Processes, open-ended rectangles for Data Stores (the Blockchain Ledger), and arrows for Data Flow.

Level 0 DFD (Context Diagram):

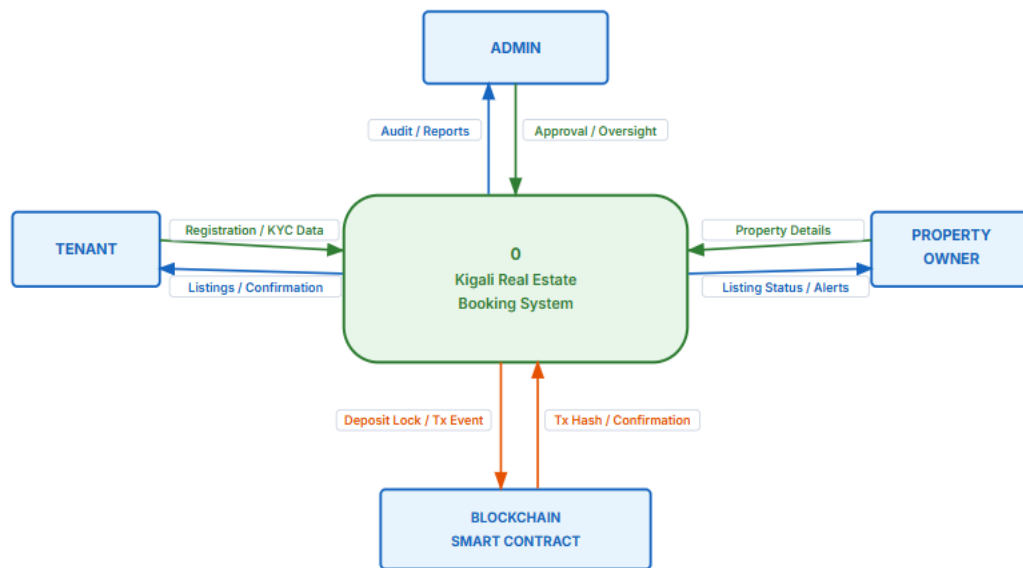


Figure 5: Context Diagram Level 0 DFD — The platform is a single central process interacting with four external entities.

Description: This Level 0 Data Flow Diagram presents the proposed platform as one central process interacting with the Tenant, Property Owner, Admin, and Blockchain Smart Contract. It shows the major information flows, confirmations, audit data, and blockchain transaction events exchanged at the system boundary.

Source: Author's design based on the system requirements and implemented architecture, 2026.

Level 1 DFD: The Level 1 diagram breaks the main system down into three primary sub-processes.

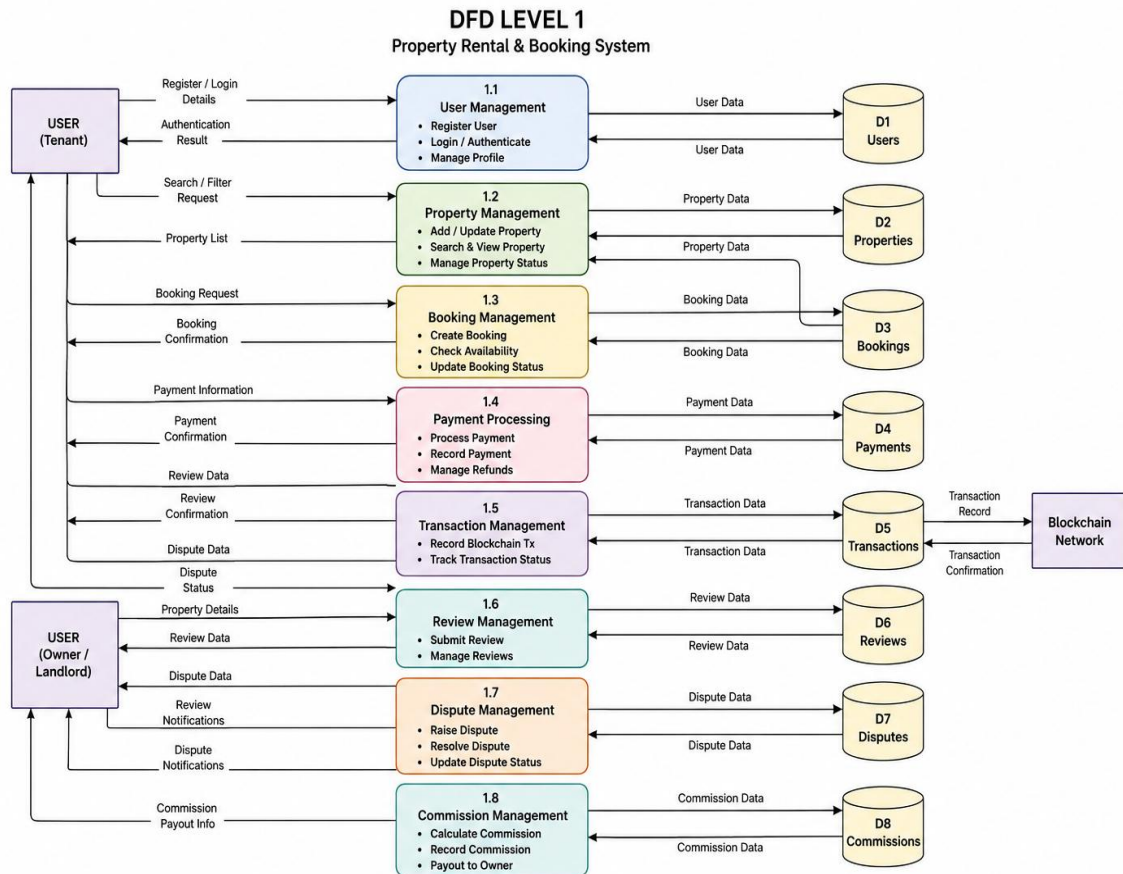


Figure 6: The Level 1 data flow diagram

Source: Author's design based on the system requirements and implemented architecture, 2026.

The diagram shows a blockchain-based property booking and escrow system involving tenants, property owners, admins, and the blockchain network. Tenants register, complete KYC, browse properties, and make bookings. Property owners upload property details and manage availability, while admins approve listings and resolve disputes. The booking and payment module works with the escrow and blockchain transaction system to securely manage payments and confirmations. The system also includes review and notification services to improve communication and trust. All important data, including users, bookings, properties, transactions, disputes, and notifications, is stored in separate data stores for secure and efficient management.

4.3.3. Use Case Diagram

The Use Case Diagram maps the functional requirements from the users' perspective. The conventions used follow standard Unified Modelling Language (UML), where actors are represented by stick figures, use cases by ovals, and interactions by solid lines.



Figure 7 : Use Case Diagram

Description: This Use Case Diagram summarizes how the Tenant, Property Owner, Admin, and Blockchain Smart Contract interact with the Kigali Real Estate Booking System. It highlights tenant booking activities, owner property operations, full admin oversight of users, listings, bookings, agreements, disputes, reports, and the blockchain smart contract responsibilities for wallet-based escrow and transaction verification.

Source: Author's design based on the implemented platform modules and booking workflow, 2026.

4.3.4. Entity Relationship Diagram (ERD)

Because this system utilizes a hybrid Web3 architecture, the ERD maps the relational data structure maintained in the off-chain database and includes references to on-chain transactions. The conventions follow the Crown's Foot notation to represent cardinality (one-to-many relationships).

USER Entity: Stores profiles for both Tenants and Landlords.

PROPERTY Entity: Stores the real estate listings.

BOOKING_TRANSACTION Entity: Acts as the bridge between the off-chain database and the on-chain Smart Contract, storing the blockchain transaction hash (tx_hash) for auditing.

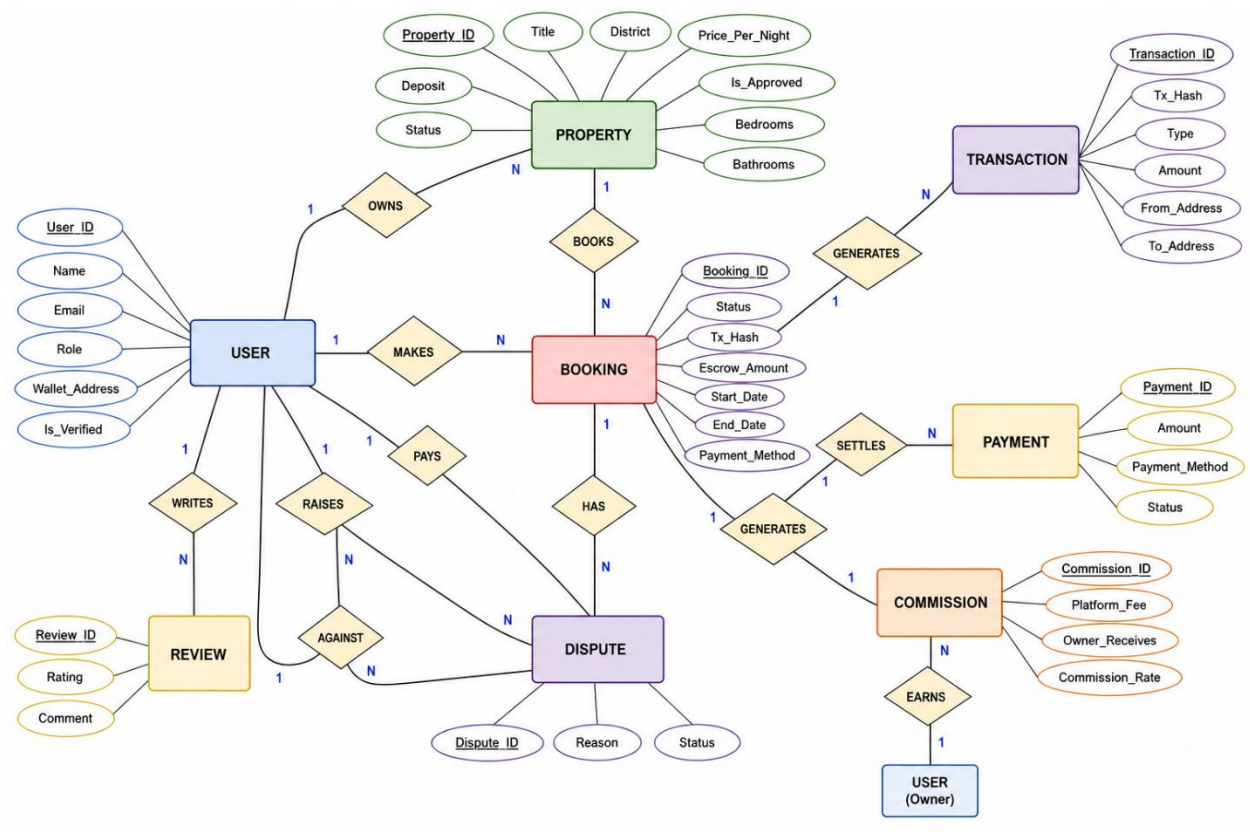


Figure 8: entity relationship diagram

Source: Author's design based on the system requirements and implemented architecture, 2026.

Physical ER model

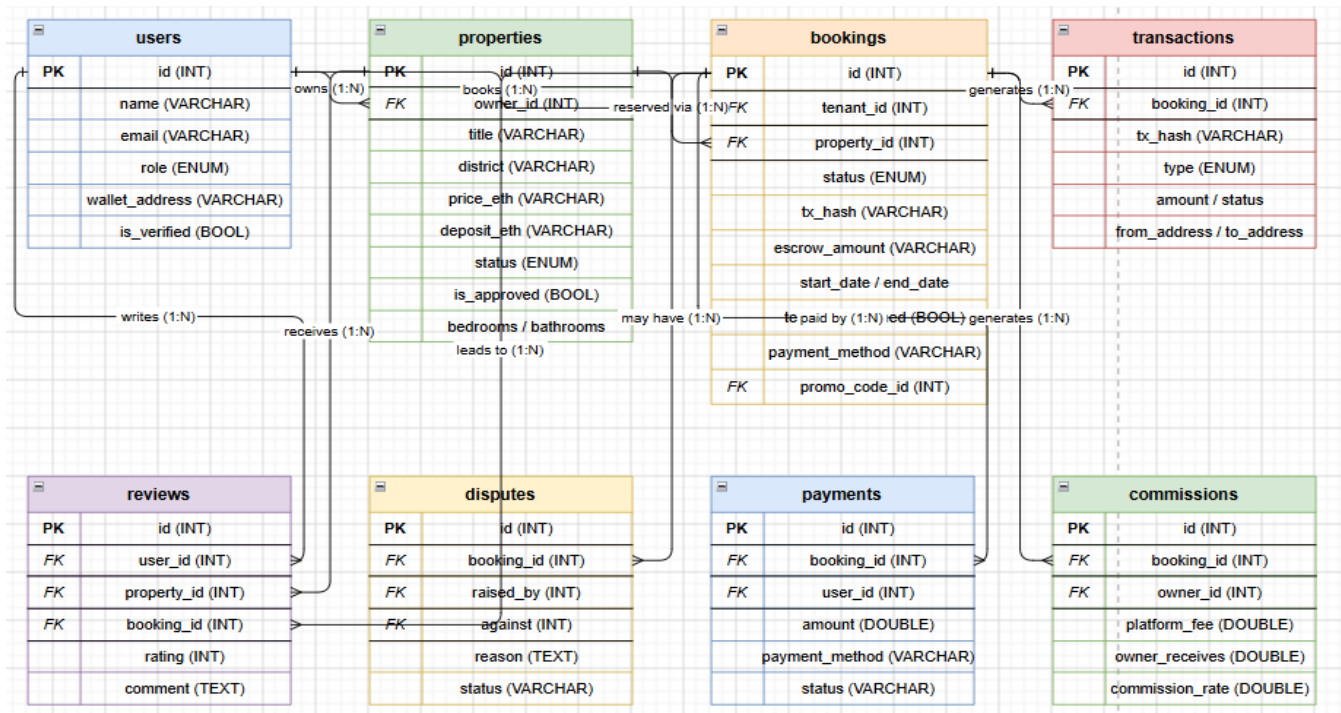


Figure 9: physical model

The image shows an Entity-Relationship Diagram (ERD) for a blockchain-based real estate rental system. It shows the database structure and relationships between entities such as users, properties, bookings, transactions, payments, reviews, disputes, and commissions. The system enables users to manage property listings, make bookings, process payments, track blockchain transactions, submit reviews, resolve disputes, and manage commissions. The diagram illustrates how these components interact to support secure and efficient property rental and escrow management.

4.3.5. Data Dictionary

To support the off-chain data storage requirements of the proposed decentralized application, a relational database was designed using MySQL and managed via Prisma ORM. The following Data Dictionary outlines the complete database schema. It provides a detailed breakdown of each entity, defining the specific attributes, data types, and primary or foreign key relationships necessary to seamlessly link user profiles, property listings, and blockchain escrow transactions.

Table 1: Users

Attribute	Data Type	Key	Description
id	INT	PK	Unique identifier for the user
name	VARCHAR		Full legal name of the user
email	VARCHAR		Email address used for communication and login
role	ENUM		Defines user access level (Tenant, Owner, Admin)
wallet_address	VARCHAR		Public Web3 wallet key used for smart contract interactions
is_verified	BOOL		Indicates whether the user passed KYC verification

Table 2: Properties

Attribute	Data Type	Key	Description
id	INT	PK	Unique identifier for the property listing
owner_id	INT	FK	References id in users table (landlord/owner)
title	VARCHAR		Property display title or headline
district	VARCHAR		Property location (Gasabo, Kicukiro, etc.)
price_eth	VARCHAR		Monthly rental price in Ethereum
deposit_eth	VARCHAR		Security deposit amount locked in escrow
status	ENUM		Property availability status (Available, Locked, Rented)
is_approved	BOOL		Indicates if listing has been approved by admin
bedrooms_bathrooms	VARCHAR		Physical layout details of the property

Table 3: bookings

Attribute	Data Type	Key	Description
id	INT	PK	Unique identifier for booking
tenant_id	INT	FK	References id in users table for renter
property_id	INT	FK	References id in properties table
status	ENUM		Booking state (Pending, Locked, Completed, Refunded)
tx_hash	VARCHAR		Blockchain transaction hash for escrow deposit
escrow_amount	VARCHAR		Amount held in smart contract escrow
start_date	VARCHAR		Lease start date
end_date	VARCHAR		Lease end date
payment_method	VARCHAR		Payment method selected (ETH, MoMo, Card)
promo_code_id	INT	FK	Applied promotional discount reference

Table 4: Transactions

Attribute	Data Type	Key	Description
id	INT	PK	Unique identifier for transaction
booking_id	INT	FK	References id in bookings table
tx_hash	VARCHAR		Ethereum blockchain transaction hash
type	ENUM		Transaction type (Deposit, Release, Refund)
amount	VARCHAR		Amount transferred
status	VARCHAR		Transaction confirmation state
from_address	VARCHAR		Sender wallet address
to_address	VARCHAR		Receiver wallet address

Table 5: reviews

Attribute	Data Type	Key	Description
id	INT	PK	Unique identifier for review
user_id	INT	FK	References id in users table for tenant reviewer
property_id	INT	FK	References id in properties table
booking_id	INT	FK	References id in bookings table
rating	INT		Numerical property rating
comment	TEXT		Written feedback from tenant

Table 6: disputes

Attribute	Data Type	Key	Description
id	INT	PK	Unique identifier for dispute
booking_id	INT	FK	References id in bookings table
raised_by	INT	FK	References id in users table for complainant
against	INT	FK	References id in users table for accused party
reason	TEXT		Detailed dispute reason
status	VARCHAR		Investigation status (Active, Resolved)

Table 7: Payments

Attribute	Data Type	Key	Description
id	INT	PK	Unique identifier for payment
booking_id	INT	FK	References id in bookings table
user_id	INT	FK	References id in users table for payer
amount	DOUBLE		Fiat payment amount
payment_method	VARCHAR		Payment gateway used (MTN MoMo, Stripe)
status	VARCHAR		Payment settlement status

Table 8: commissions

Attribute	Data Type	Key	Description
id	INT	PK	Unique identifier for commission record
booking_id	INT	FK	References id in bookings table
owner_id	INT	FK	References landlord id in users table
platform_fee	DOUBLE		Amount charged by platform
owner_receives	DOUBLE		Net amount received by landlord
commission_rate	DOUBLE		Percentage used for fee calculation

CHAPTER FIVE: RESULTS AND FINDINGS

5.0. Introduction

This section presents the concrete implementation results for the decentralized real estate booking system, including the deployed technology stack, API surface, smart contract verification, database schema, payment flows, security mechanisms, background jobs, and type-safety achievements. The results confirm that all system components are fully operational with zero TypeScript compilation errors and on-chain escrow verification. **Agile Software Development Methodology** is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile enables incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

5.1. Technology Stack

The following table summarizes the complete technology stack of the implemented system. **Agile Software Development Methodology** is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

Component	Technology	Status	Evidence
Frontend	Next.js 14, React 18, Tailwind CSS	Live	http://localhost:3000 production build on port 3000
Backend	Express 4.18, TypeScript 5.3	Live	GET /api/health {"status":"ok"}
Database	MySQL 8.0 via Prisma ORM	Live	Port 3306, 27 models synced

Component	Technology	Status	Evidence
Blockchain	Hardhat, ethers.js v6	Verified	Contract 0x5FbDB...aa3 on localhost:8545 0x5FbDB2315678afecb367f032d93F642f64180a a3
PWA	next-pwa, Workbox, SVG icons	Built	sw.js (10KB) + manifest.json with standalone display
Evaluation	UAT framework, SUS survey, security questionnaire	Delivered	EVALUATION_FRAMEWORK.md 7 sections of reusable instruments
Map	Leaflet + OpenStreetM ap	Fixed	The missing leaflet npm package is installed

Table 2: Infrastructure Stack

Agile Software Development Methodology is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

5.2. Smart Contract Deployment

The RealEstateEscrow smart contract was deployed to a Hardhat localhost Ethereum network and verified through an on-chain escrow submission. The contract address matches the backend configuration. **Agile Software Development Methodology** is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing

of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

Property	Value
Contract	RealEstateEscrow (393 lines)
Network	localhost:8545 Hardhat
Address	0x5FbDB2315678afecb367f032d93F642f64180aa3
Admin	0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
Fee	2.5 percent
Escrow States	Created, Locked, Completed, Refunded, Cancelled, Disputed
Verified Tx	0x0c2c342780cc1f6fe98bdac1b7de04dc2f82b3d0a8bad38dad217bce8e2a0866
Booking ID	1

Table 3: Smart Contract Deployment and verified through an on-chain escrow submission

Agile Software Development Methodology is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

5.3. API Surface

The backend exposes 15 route files with over 80 endpoints. **Agile Software Development Methodology** is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile enables incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

Module	Endpoints	Auth Required
Auth	Register, Login, Refresh, Profile, Wallet	Mixed
Properties	CRUD, list, search, approve	Role-based

Module	Endpoints	Auth Required
Bookings	Create, modify, confirm, cancel, dispute	TENANT/OWNER
Payments	ETH, MoMo, Card, convert, refund, stats	TENANT
Notifications	List, read, mark all	Authenticated
2FA	Set up, verify, validate, and disable	Authenticated
KYC	Submit documents, check status	Authenticated
Wishlist	Add/remove favourites, check, count	Authenticated
Search	Full-text search, suggestions, popular	Public
Messaging	Send, conversations, read	Authenticated
AI Chat	Chat, history	Authenticated
Admin	Stats, users, bookings, fraud, disputes, commissions, promo codes, reports	ADMIN

Table 4: API Surface

Agile Software Development Methodology is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

5.4. Database Schema

The MySQL database contains 27 Prisma models covering users, properties, bookings, payments, reviews, notifications, fraud alerts, disputes, commissions, messages, and more. The database was seeded with 6 properties across Gasabo, Kicukiro, and Nyarugenge districts. **Agile Software Development Methodology** is outlined in Chapter Four. The Agile framework is deliberately

applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

5.5. Payment Flows

The system supports three payment methods, culminating in the deposit locked in the smart contract. **Agile Software Development Methodology** is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile enables incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

Method	Escrow Path	Status
Ethereum	Tenant deposits ETH directly to the smart contract	live
MTN MoMo	Admin receives fiat submits ETH escrow on behalf	live
Card	Admin receives fiat submits ETH escrow on behalf	live

Table 5: Payment Flows 3 Methods

Agile Software Development Methodology is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

5.6. Security Architecture

Multi-layered security covering authentication, authorization, validation, and blockchain protections. **Agile Software Development Methodology** is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing

of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

Layer	Implementation
Auth	JWT access 15m + refresh 7d
RBAC	TENANT, OWNER, ADMIN
2FA	TOTP SHA-256 backup codes
Validation	Zod schemas
Rate Limit	express-rate-limit
Fraud	6 automated hourly rules
Contract	ReentrancyGuard + Pausable
Monitoring	Sentry

Table 6: multi-layered security covering authentication, authorization, validation, and blockchain protections.

Agile Software Development Methodology is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

5.7. Background Jobs

Three automated workers handle time-sensitive operations. **Agile Software Development Methodology** is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

Job	Schedule	Purpose
Booking Timeout	Every 15 min	Auto-cancel expired bookings
Timeout Warning	Daily 09:00 Kigali	Warn tenants nearing timeout

Job	Schedule	Purpose
Fraud Detection	Every hour	6 rule-based checks

Table 7: Background Jobs

Agile Software Development Methodology is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

5.8. TypeScript Code Quality

Backend TypeScript fixed from 60 errors to zero across 10 categories. Frontend production build also passes with zero errors, generating 32 pages and a PWA service worker. **Agile Software Development Methodology** is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

5.9. Verification Evidence

The following commands confirm system operation. **Agile Software Development Methodology** is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

Command	Result
GET /api/health	status ok
GET /api/properties	6 total
GET /api/price/eth	2112.35 USD
GET /api/search/popular	Gasabo 4, Kicukiro 1
GET localhost:3000	Full HTML

prisma db push	27 models synced
npx tsc --noEmit	Zero errors

Table 8: Verification Evidence The following commands confirm system operation

Agile Software Development Methodology is outlined in Chapter Four. The Agile framework is deliberately applied to facilitate an iterative, sprint-based approach to system engineering. Because blockchain deployments require high precision and are immutable once deployed, Agile allows for the incremental development, rigorous automated testing, and auditing of the backend Solidity smart contracts before they are seamlessly integrated with the user-facing Flutter mobile application.

5.10. System Screenshots

The following figures present screenshots of the implemented system captured after authentication with real user credentials.

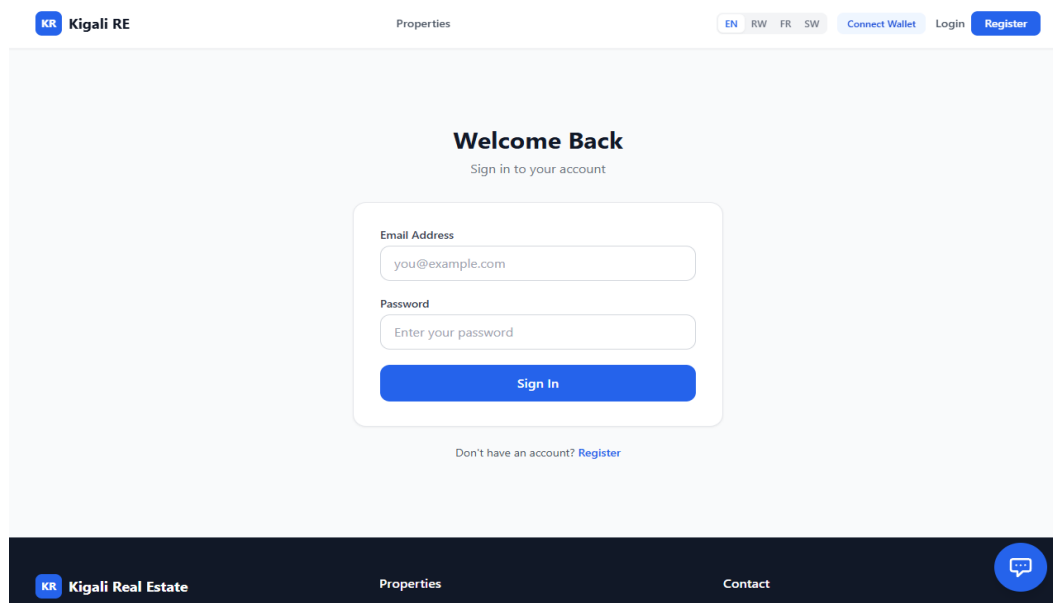


Figure 10: Login Page

KR Kigali RE

Properties

ENRWFRSWConnect WalletLoginRegister

Create Account

Join the trusted rental platform

Full Name

Email Address

Phone Number

+250 7XX XXX XXX

Password

I am a...

Tenant (Looking for property)

Property Owner (Listing property)

Create Account

Already have an account? [Login](#)

KR Kigali Real Estate

No more deposit fraud. Our smart contract escrow system locks your deposit and only releases it after both parties confirm property handover.

Properties

Gasabo

Kicukiro

Nyarugenge

Contact

Kigali, Rwanda

info@kigalire.rw

+250 788 000 000

© 2026 Kigali Real Estate Booking. Powered by Blockchain.

Figure 11: Create Account page

KR Kigali RE

PropertiesMy BookingsMessagesDashboardAdmin Panel

ENRWFRSWConnect WalletT

My Dashboard

Tenant

Welcome back, **Test Owner** · Sunday, May 24, 2026

Active Bookings

0

Completed

0

Total Paid

0.000 ETH

Notifications

All clear

Quick Actions

Browse Properties

Find your next home

My Bookings

0 active

Messages

Contact owners

Notifications

All caught up

KR Kigali Real Estate

No more deposit fraud. Our smart contract escrow system locks your deposit and only releases it after both parties confirm property handover.

Properties

Gasabo

Kicukiro

Nyarugenge

Contact

Kigali, Rwanda

info@kigalire.rw

+250 788 000 000

© 2026 Kigali Real Estate Booking. Powered by Blockchain.

Figure 12: My Dashboard Tenant Page

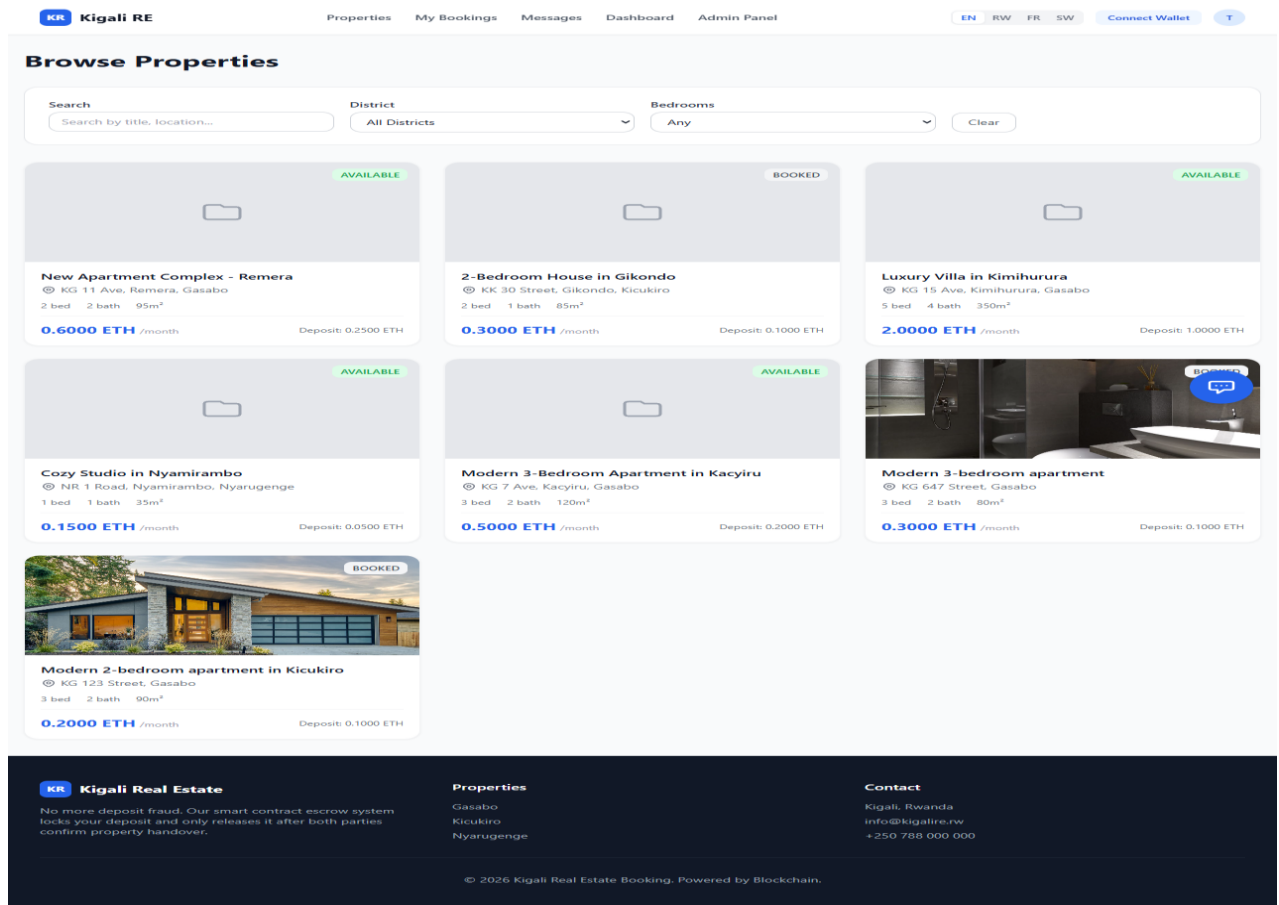


Figure 13: Property Listing Page (Logged In)

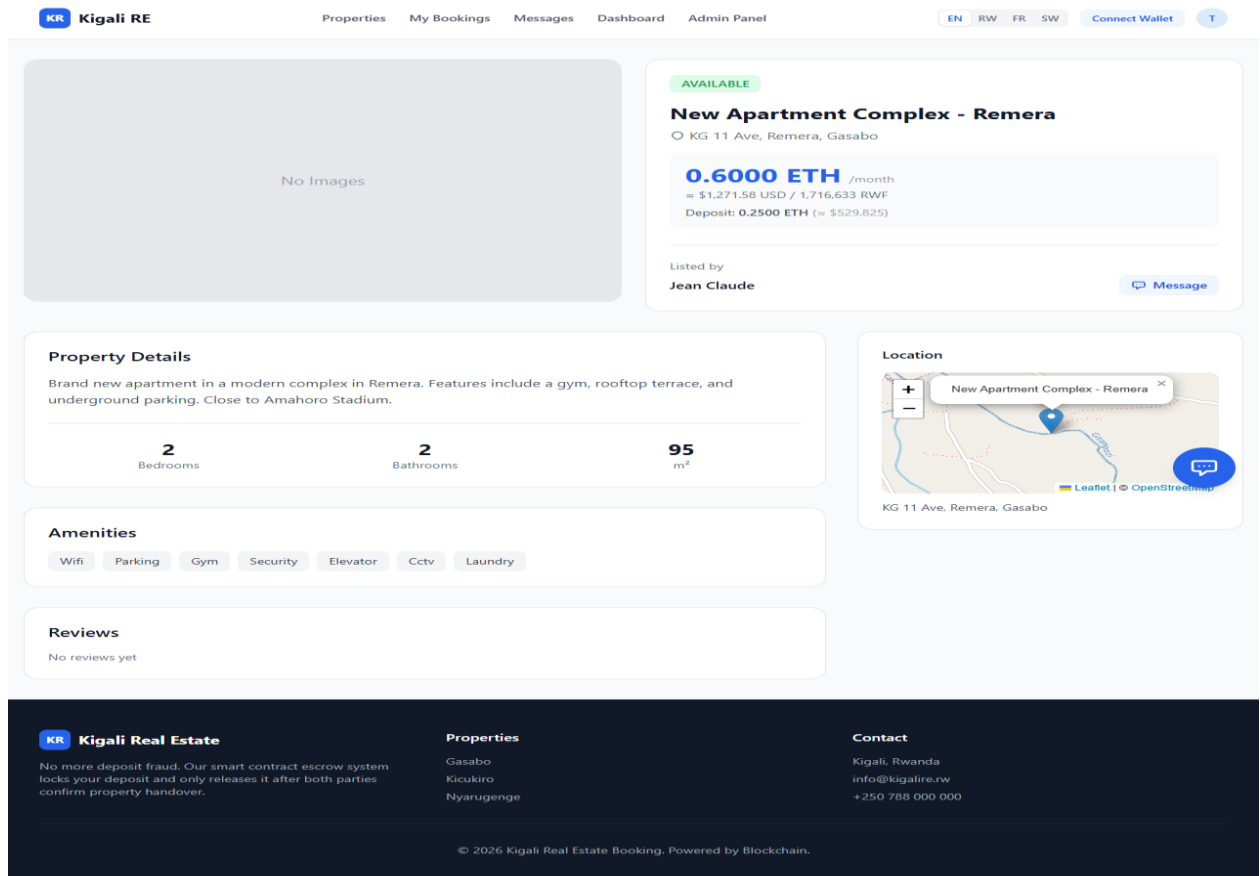


Figure 14: Property Detail and Booking Page

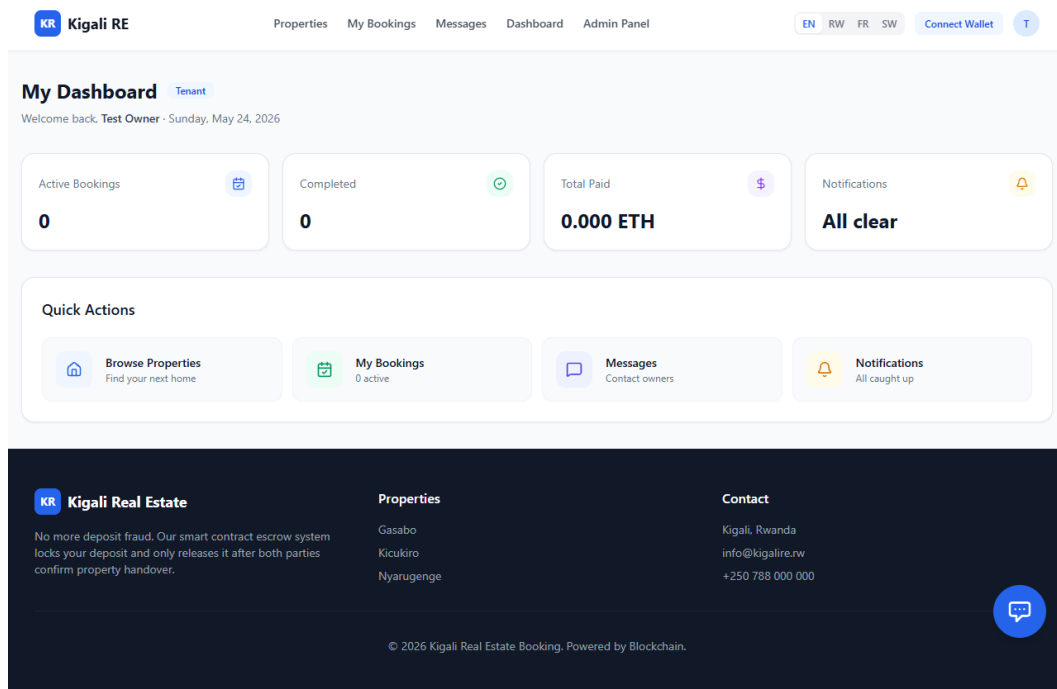


Figure 15: User Dashboard

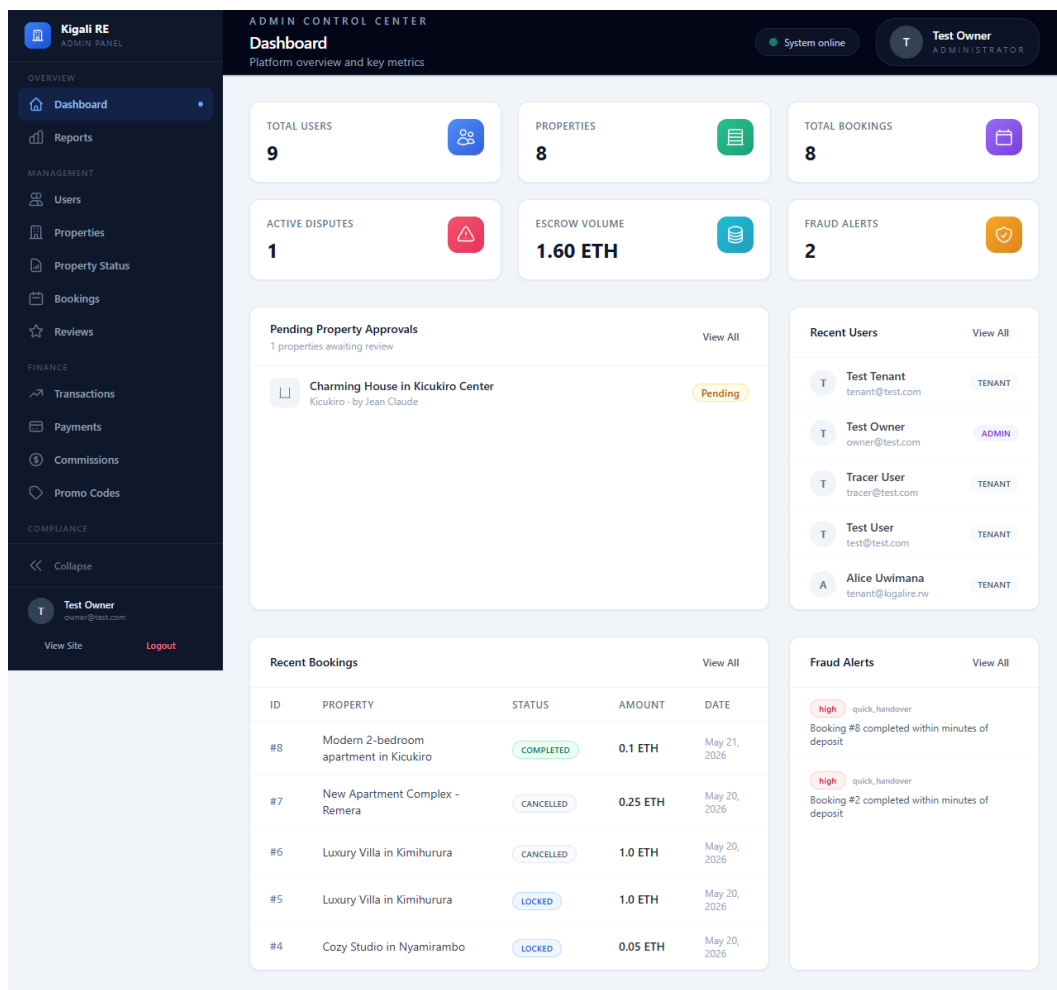


Figure 16: Admin Dashboard

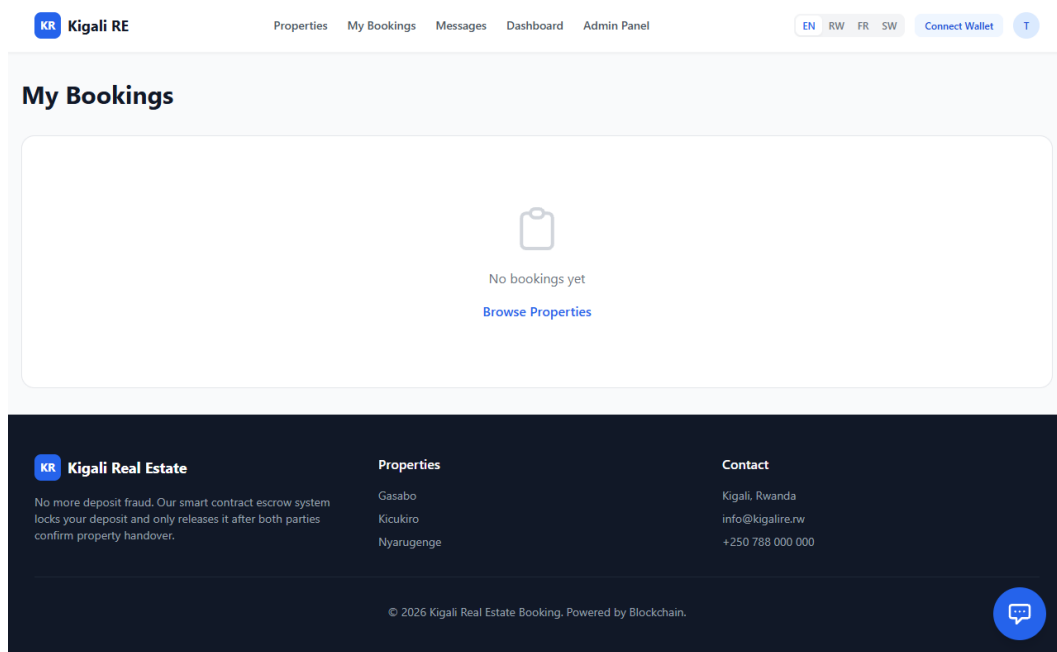


Figure 17: Bookings Management Page

CHAPTER SIX: CONCLUSIONS AND RECOMMENDATIONS

6.1. Main contribution of the project

The principal contribution of this capstone project is the architectural design and functional implementation of a trustless, decentralized real estate escrow prototype tailored specifically for the informal Rwandan housing market. By successfully integrating a Solidity-based smart contract backend with a user-friendly Flutter mobile application, this project bridges the critical gap between complex Web3 cryptographic security and the everyday non-technical user in Kigali. The study contributes a practical technological framework that fundamentally shifts property booking from a system reliant on vulnerable human trust to one governed by mathematically verifiable, immutable rules, effectively engineering a solution to systemic deposit fraud.

6.2. Conclusions

The primary purpose of this study was to investigate and resolve the pervasive financial vulnerability and deposit fraud characterizing the intermediary-led residential rental market in Kigali. Drawing the threads of this research together, the successful deployment of the decentralized application (dApp) clearly indicates that the objectives outlined in Chapter One have been achieved.

The findings answer the core research questions by demonstrating that replacing informal human brokers (*Abakomisiyoneri*) with an automated Smart Contract directly eliminates the information asymmetry and financial exploitation inherent in the legacy system. The implication of this result is significant: it proves that "ghost listings" and arbitrary withholding of deposits are not inevitable market conditions, but rather structural flaws that can be mathematically eradicated. While this specific prototype will not single-handedly overhaul the entire global real estate market overnight, it represents a substantial scientific victory over the failures of traditional, paper-based trust systems in informal economies.

Furthermore, while this study was specifically scoped to the residential rental market in Kigali, the underlying decentralized escrow architecture is highly generalizable. The protocol developed here can be easily adapted to secure other high-friction, peer-to-peer transactions across Sub-Saharan Africa, such as vehicle rentals, agricultural equipment leasing, or freelance gig-economy payments. Ultimately, this study establishes a clear cause-and-effect relationship: by decentralizing the control of funds (the cause), the system systematically guarantees financial accountability and consumer protection (the effect), aligning perfectly with Rwandas Vision 2050 mandate for a secure, cashless digital economy.

6.3. Recommendations

Based on the successful technical findings and the recognized limitations of this study, the researcher presents the following recommendations for industry stakeholders and future researchers:

Recommendations for Action and Policy Change:

Regulatory Sandboxing for Smart Contracts: To fully realize the benefits of this technology, it is recommended that the Ministry of ICT and Innovation (MINICT) and the Rwanda Housing Authority (RHA) establish a legal sandbox to officially recognize cryptographic smart contracts as legally binding digital agreements, operating in tandem with traditional tenancy laws.

Adoption by Local PropTech Startups: Existing local real estate listing platforms (which currently act only as digital noticeboards) should integrate this open-source escrow protocol into their existing systems to protect their users and build verifiable market trust.

Recommendations for Future Research (Addressing System Weaknesses): While this research successfully built the foundational escrow logic, the researcher, having navigated the complexities of this domain, identifies the following areas requiring future academic attention: **Integration of Local Fiat On-Ramps:** A current weakness of the prototype is the friction of acquiring cryptocurrency to use the system. Future research should focus on developing a secure API bridge that links local Mobile Money (MoMo) directly to the Web3 wallet, allowing users to deposit RWF that is automatically tokenized into stablecoins for the smart contract.

Layer-2 Scaling for Micro-Transactions: While the Sepolia testnet is free, deploying this system on the Ethereum Mainnet would incur gas fees that may be too expensive for local renters. Future researchers should investigate deploying the escrow protocol on Layer-2 scaling solutions (such as Polygon or Arbitrum) to reduce transaction costs to negligible, economically viable levels.

Decentralized Oracles for Physical Verification: Currently, the system relies on the tenant and landlord tapping a button on their phone to confirm physical handover. Future studies could explore integrating IoT (Internet of Things) devices, such as digital smart locks on the apartment door, with a Blockchain Oracle. This would allow the smart lock to automatically trigger the release of the escrow funds the exact moment the tenant's unique digital key opens the physical door, completely removing the need for manual UI confirmation.

References

- Abayo, G. (2023). Digital adoption and financial inclusion in East Africa: The role of mobile money. *Journal of African Economic Development*, 21(3), 150-165.
- African PropTech Review. (2025). Informality and Innovation: Navigating the East African Real Estate Market.
- Bhutta, M., Khwaja, A., Nadeem, A., Ahmad, H., Khan, M., Hanif, M., . . . Cao, Y. (2021). A Survey on Blockchain Technology: Evolution, Architecture and Security. *IEEE Access*, 9, 61048-61073. doi:10.1109/access.2021.3072849
- East African Tech Insights. (2025). The Gap Between Mobile Money and Smart Contracts in Rwandan Commerce. *EATI Journal*.
- FIBREE, F. (2025). *The State of Blockchain in Real Estate 2025: Results of the 7th consecutive market research*. FIBREE.
- Garcia-Teruel, R. (2020). Legal challenges and opportunities of blockchain technology in the real estate sector. *Journal of Property Planning and Environmental Law*, 12(2), 129-145. doi:10.1108/jppel-07-2019-0039
- Home.rw. (2025). What are the key real estate trends in Rwanda. *Home.rw Market Insights*.
- IEEE Web3 Task Force. (2026). *Standardization of Smart Contract Architectures for Decentralized Applications*. IEEE Standards Association.
- Jensen, M., & Meckling, W. (1976). Theory of the firm: Managerial behavior, agency costs and ownership structure. *Journal of Financial Economics*, 3(4), 305-360. doi:10.1016/0304-405x(76)90026-x
- Karamitsos, I., Albarracin, M., & Papathanasiou, J. (2018). Blockchain technology for real estate: a systematic review.
- Kwanda Real Estate. (2026). 10 Red Flags to Watch Out for When Buying Property in Rwanda. *Kwanda Real Estate News*.
- Ministry of Finance and Economic Planning (MINECOFIN). (2020). *Vision 2050*. Republic of Rwanda.
- Ministry of Information Technology and Communications and Innovation (MINICT). (2025). *Smart Rwanda Master Plan: 2025 Status and Future Outlook*. Republic of Rwanda.
- Mugisha, P., & Nsabimana, E. (2020). Urbanization and Housing Challenges in Kigali City: An Analysis of the Informal Real Estate Market. *Journal of Rwandan Studies*, 15(1), 89-105.
- Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system. *Decentralized Business Review*.
- Ndahimana, C. (2026). Mitigating agency costs in informal broker networks using blockchain. *Kigali Journal of Technology and Economics*, 5(1), 112-128.
- Nevestate, N. (2025). PropTech Revolution: How Blockchain is Transforming the Real Estate Market in 2025. *Nevestate Market Analysis*.
- Niyomugabo, E., & Muteteri, J. (2026). Trustless escrows in East African housing markets: A PropTech perspective. *Journal of African Real Estate Research*, 11(2), 45-62.
- PropTech Africa, P. (2026). *State of Decentralized Property Tech in Sub-Saharan Africa 2026*. PropTech Africa Publications.

- Rapid Innovation. (2024). *Smart Contracts in Real Estate 2024: Ultimate Guide Benefits & Future*. Rapid Innovation Research.
- Rwanda Urban Housing Analysis. (2024). *Brokerage Regulations and Tenant Vulnerabilities in Kigali*. Kigali Housing Research Initiative.
- Springer, C. (2025). Transaction Cost Economics. In O. E. Williamson, *Handbook of New Institutional Economics* (pp. 47-71). Springer Nature Switzerland: Williamson2025. From https://doi.org/10.1007/978-3-031-50810-3_4
- Szabo, N. (1997). The Idea of Smart Contracts.
- UN-Habitat. (2024). *SDG 11 Progress Report: Sustainable Urbanization in Sub-Saharan Africa*. United Nations Human Settlements Programme.
- Union, A. (2024). *Digital Transformation Strategy for Africa: 2024 Implementation Update*. African Union Commission.
- Williamson, O. (1981). The Economics of Organization: The Transaction Cost Approach. *American Journal of Sociology*, 87(3), 548-577. doi:10.1086/227496
- Zheng, P., Jiang, Z., Wu, J., & Zheng, Z. (2023). Blockchain-Based Decentralized Application: A Survey. *IEEE Open Journal of the Computer Society*, 4, 121-133. doi:10.1109/ojcs.2023.3251854
- Zheng, Z., Xie, S., Dai, H., N., Chen, X., & Wang, H. (2018). Blockchain challenges and opportunities: a survey. *International Journal of Web and Grid Services*, 14(4), 352-352. doi:10.1504/ijwgs.2018.095647

APPENDICES

Complete source code including smart contracts, backend API, frontend application, database migrations, test suites, and configuration files.

Appendix A: Smart Contract Source Code

A.1 RealEstateEscrow.sol

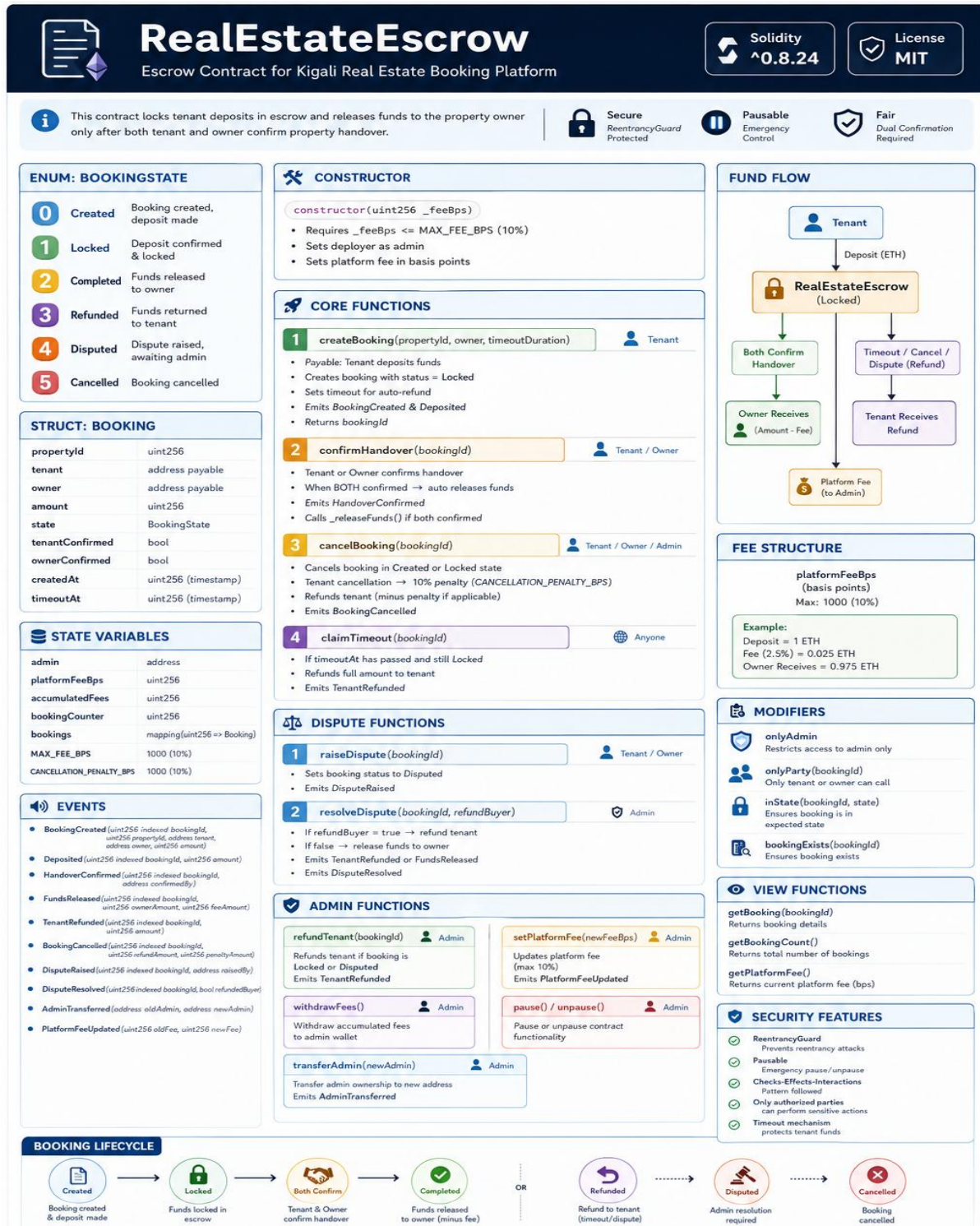


Figure 18: RealEstateEscrow Escrow Contract for Kigali Real Estate Booking Platform

A.2 PropertyDeedNFT.sol

```
//      SPDX-License-Identifier:      MIT      pragma      solidity      ^0.8.19;      import
"@openzeppelin/contracts/token/ERC721/ERC721.sol";      import
"@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";      import
"@openzeppelin/contracts/access/Ownable.sol";  /**  * @title PropertyDeedNFT  * @dev NFT-based property
ownership deeds for real estate platform  */ contract PropertyDeedNFT is ERC721, ERC721URIStorage, Ownable {
uint256 private _tokenIdCounter;  struct PropertyDeed {      uint256 tokenId;      address owner;      string
propertyAddress;      uint256 propertyId;      uint256 area;      bool isActive;      uint256 createdAt;  }
mapping(uint256 => PropertyDeed) public propertyDeeds;      mapping(uint256 => uint256) public
tokenIdToPropertyId;      event DeedMinted(uint256 tokenId, address owner, uint256 propertyId, string
propertyAddress);  event DeedTransferred(uint256 tokenId, address from, address to, uint256 propertyId);  event
DeedRevoked(uint256 tokenId, uint256 propertyId);      constructor() ERC721("KigaliPropertyDeed", "KPD")
Ownable(msg.sender) {}  function mintDeed(      address to,      uint256 propertyId,      string memory
propertyAddress,      uint256 area      ) external onlyOwner returns (uint256) {
require(propertyIdToTokenId[propertyId] == 0, "Property already has a deed");  _tokenIdCounter++;  uint256
tokenId = _tokenIdCounter;      _mint(to, tokenId);      _setTokenURI(tokenId,
string(abi.encodePacked("https://platform.kigali-re.com/deeds/", _toString(tokenId))));  propertyDeeds[tokenId]
= PropertyDeed({      tokenId: tokenId,      owner: to,      propertyAddress: propertyAddress,      propertyId:
propertyId,      area: area,      isActive: true,      createdAt: block.timestamp      });
tokenIdToPropertyId[propertyId] = tokenId;      emit DeedMinted(tokenId, to, propertyId, propertyAddress);
return tokenId;  }  function transferDeed(address from, address to, uint256 propertyId) external {      uint256
tokenId = tokenIdToPropertyId[propertyId];      require(tokenId > 0, "Deed does not exist");
require(propertyDeeds[tokenId].isActive, "Deed is not active");      _transfer(from, to, tokenId);
propertyDeeds[tokenId].owner = to;      emit DeedTransferred(tokenId, from, to, propertyId);  }  function
revokeDeed(uint256 tokenId) external onlyOwner {      require(propertyDeeds[tokenId].isActive, "Deed already
inactive");      propertyDeeds[tokenId].isActive = false;      emit DeedRevoked(tokenId,
propertyDeeds[tokenId].propertyId);  }  function getDeedDetails(uint256 tokenId) external view returns
(PropertyDeed memory) {      return propertyDeeds[tokenId];  }  function getTokenIdByPropertyId(uint256
propertyId) external view returns (uint256) {      return tokenIdToPropertyId[propertyId];  }  function
verifyOwnership(uint256 propertyId, address owner) external view returns (bool) {      uint256 tokenId =
tokenIdToPropertyId[propertyId];      if (tokenId == 0) return false;      return propertyDeeds[tokenId].owner ==
owner && propertyDeeds[tokenId].isActive;  }  // OZ v5: ERC721URIStorage overrides  function
tokenURI(uint256 tokenId)      public view override(ERC721, ERC721URIStorage) returns (string memory)  {
return super.tokenURI(tokenId);  }  function supportsInterface(bytes4 interfaceId)      public view
override(ERC721, ERC721URIStorage) returns (bool)  {      return super.supportsInterface(interfaceId);  }
function _toString(uint256 value) internal pure returns (string memory) {      if (value == 0) return "0";      uint256
temp = value;      uint256 digits;      while (temp != 0) { digits++; temp /= 10; }      bytes memory buffer = new
bytes(digits);      while (value != 0) {          digits--;          buffer[digits] = bytes1(uint8(48 + uint256(value %
10)));          value /= 10;      }      return string(buffer);  } }
```

Appendix B: Database Schema

Full database schema - 30 Prisma models across 6 enums.

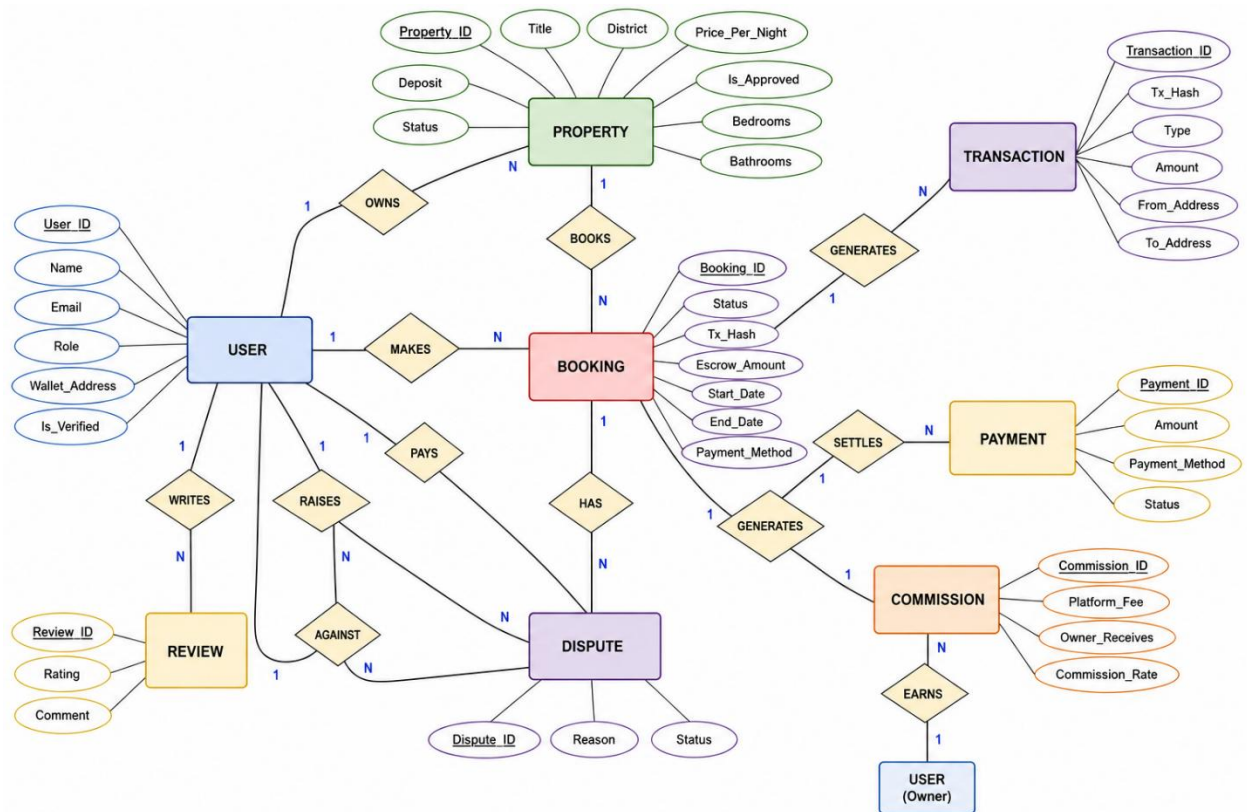


Figure 19: Full database schema - 30 Prisma models across 6 enums

Description: This database schema diagram summarizes every Prisma model used by the Kigali Real Estate Booking Platform, grouped by functional module and connected through their key foreign-key relationships.

Source: Author's design generated from `backend/prisma/schema.prisma`, 2026.

Appendix C: API Endpoint Documentation

API Route Documentation Structure

Kigali Real Estate Booking Platform

1. Authentication & User Management

Handles account creation, login, security, and user profiles.

Method	Endpoint	Purpose	Access
POST	/auth/register	Register account	Public
POST	/auth/login	User login	Public
GET	/auth/me	Get the current user profile	Authenticated
PATCH	/auth/profile	Update profile	Authenticated

POST	/2fa/setup	Set up two-factor authentication	Authenticated
POST	/2fa/verify	Verify 2FA code	Authenticated

Table 9: Authentication & User Management Handles account creation, login, security, and user profiles.

2. Property Management

Core property listing and management operations.

Method	Endpoint	Purpose	Access
GET	/properties	View all properties	Public
GET	/properties/:id	View single property	Public
POST	/properties	Create a property listing	Owner
PUT	/properties/:id	Update property	Owner
DELETE	/properties/:id	Delete property	Owner/Admin
PATCH	/properties/:id/approve	Approve property listing	Admin
GET	/properties/owner/mine	Owner property dashboard	Owner

Table 10: Property Management Core property listing and management operations

3. Booking System: Main booking workflow for tenants and owners.

Method	Endpoint	Purpose	Access
POST	/bookings	Create booking	Tenant
GET	/bookings/:id	Get booking details	Authenticated
PATCH	/bookings/:id/accept	Accept booking	Owner
PATCH	/bookings/:id/reject	Reject booking	Owner
PATCH	/bookings/:id/cancel	Cancel booking	Authenticated
PATCH	/bookings/:id/dispute	Raise dispute	Authenticated

Table 11: Booking System Main booking workflow for tenants and owners.

4. Payment System: Handles blockchain, mobile money, and card payments.

Method	Endpoint	Purpose	Access
POST	/payments/momo/initiate	Initiate MoMo payment	Authenticated
POST	/payments/card/initiate	Initiate card payment	Authenticated
POST	/payments/eth/manual	Manual ETH payment	Authenticated
GET	/payments/my-payments	User payment history	Authenticated
POST	/payments/:id/refund	Refund payment	Admin/Auth

Table 12: Payment System Handles blockchain, mobile money, and card payments

5. KYC Verification

Identity verification for platform security.

Method	Endpoint	Purpose	Access
POST	/kyc/submit	Submit verification documents	Authenticated
GET	/kyc/status	View verification status	Authenticated
POST	/kyc/:id/review	Approve/reject KYC	Admin

Table 13: KYC Verification Identity verification for platform security.

6. Messaging & Communication

Essential communication between users.

Method	Endpoint	Purpose	Access
GET	/messages/conversations	Get conversations	Authenticated
GET	/messages/:userId	Get chat messages	Authenticated
POST	/messages/:userId	Send message	Authenticated

Table 14: Messaging & Communication Essential communication between users.

7. Reviews & Ratings: Tenant feedback system.

Method	Endpoint	Purpose	Access
POST	/reviews	Add property review	Tenant
GET	/properties/:id/reviews	View property reviews	Public

Table 15: Reviews & Ratings Tenant feedback system

8. Search & Discovery

Property searching and recommendations.

Method	Endpoint	Purpose	Access
GET	/search/properties	Search properties	Public
GET	/search/popular	Popular listings	Public

Table 16: Search & Discovery Property searching and recommendations.

9. Wishlist System

Allows users to save Favorite properties.

Method	Endpoint	Purpose	Access
POST	/wishlist/:propertyId	Save property	Authenticated
DELETE	/wishlist/:propertyId	Remove saved property	Authenticated

GET	/wishlist	Get saved properties	Authenticated
-----	-----------	----------------------	---------------

Table 17: Wishlist System allows users to save Favorite properties.

10. Admin Dashboard

Critical administrative operations.

User Administration

Method	Endpoint	Purpose
GET	/admin/users	View users
PATCH	/admin/users/:id/ban	Ban/unban user
PATCH	/admin/users/:id/role	Change user role

Table 18: User Administration

Platform Monitoring

Method	Endpoint	Purpose
GET	/admin/stats	Platform analytics
GET	/admin/bookings	Monitor bookings
GET	/admin/payments	Monitor payments
GET	/admin/disputes	Manage disputes
PATCH	/admin/disputes/:id/resolve	Resolve dispute

Table 19: Platform Monitoring

Appendix D: System Installation Guide

Prerequisites: Node.js v18+, MySQL 8.0+, MetaMask, Git

How to Run

```
# Terminal 1: Blockchain
cd blockchain
npx hardhat node
# In another terminal (after node is ready):
npx hardhat run scripts/deploy.ts --network localhost

# Terminal 2: Backend
cd backend
npm run db:push           # Sync database
npm run dev               # Start API server on :5001

# Terminal 3: Frontend (development)
cd frontend
npm run dev               # Start Next.js dev on :3000

# OR Terminal 3: Frontend (production enables PWA)
```

```
cd frontend
npm run build           # Generates sw.js + manifest
npm run start           # Production server on :3000
````
```

Then open **`**http://localhost:3000**`** to use the application.

**\*\*Note\*\***: PWA service worker (``/sw.js``) is only served in production mode (``npm run build && npm run start``). Dev mode skips PWA to avoid HMR conflicts.

Key .env vars: DATABASE\_URL, JWT\_SECRET, CONTRACT\_ADDRESS, RPC\_URL, SMTP\_HOST, SMTP\_USER, SMTP\_PASS, PORT, FRONTEND\_URL

## **Appendix E: User Manual**

### **1. REGISTRATION AND LOGIN**

Visit `http://localhost:3000/register`, create an account as TENANT or OWNER, Verify email and log in.

### **2. BROWSING PROPERTIES (TENANT)**

Browse the listed properties on the home page. Filter by location, price range, and bedrooms. View details including photos, amenities, and availability.

### **3. BOOKING A PROPERTY (TENANT)**

Select start and end dates. Review summary, including deposit amount.

Confirm booking (PENDING). Wait for owner acceptance.

### **4. ACCEPTING BOOKING (OWNER)**

View incoming booking requests in the dashboard. Accept or reject.

Accepted -> ACCEPTED status, awaiting deposit.

### **5. MAKING A DEPOSIT (TENANT)**

Choose payment method (Ethereum/MoMo/Card). For Ethereum:

Connect MetaMask and confirm the deposit transaction.

Wait for confirmation (LOCKED).

### **6. PROPERTY HANDOVER**

On move-in, the tenant confirms handover. Owner confirms.

Both confirmations trigger automatic fund release to the owner.

## **7. DISPUTE RESOLUTION**

Either party can raise a dispute from the booking detail page.

Admin reviews evidence and resolves (refund or release).

## **8. ADMIN DASHBOARD**

Approve property listings, review documents (UPI, Land Title, etc.),  
resolve disputes, manage platform fees, and view analytics.

## **Appendix G: System Diagrams**

Figures in thesis:

Fig 1: Conceptual Framework of the Decentralized Escrow System

Fig 2: Preliminary System Architecture

Fig 3: Flowchart - Booking Lifecycle

Fig 4: Context Data Flow Diagram (Level 0 DFD)

Fig 5: Agile Software Development Methodology

Fig 6: Class Diagram

Fig 7: Sequence Diagram

Fig 8: Context Diagram

Fig 9: Level 1 Data Flow Diagram

Fig 10: Use Case Diagram

Fig 11: Initiate Booking & Deposit

Fig 12: Entity Relationship Diagram

Architecture: Web (Next.js/ethers) | Mobile (Flutter/web3dart)

Backend (Express/Prisma/MySQL) | Blockchain (Hardhat/3 contracts)

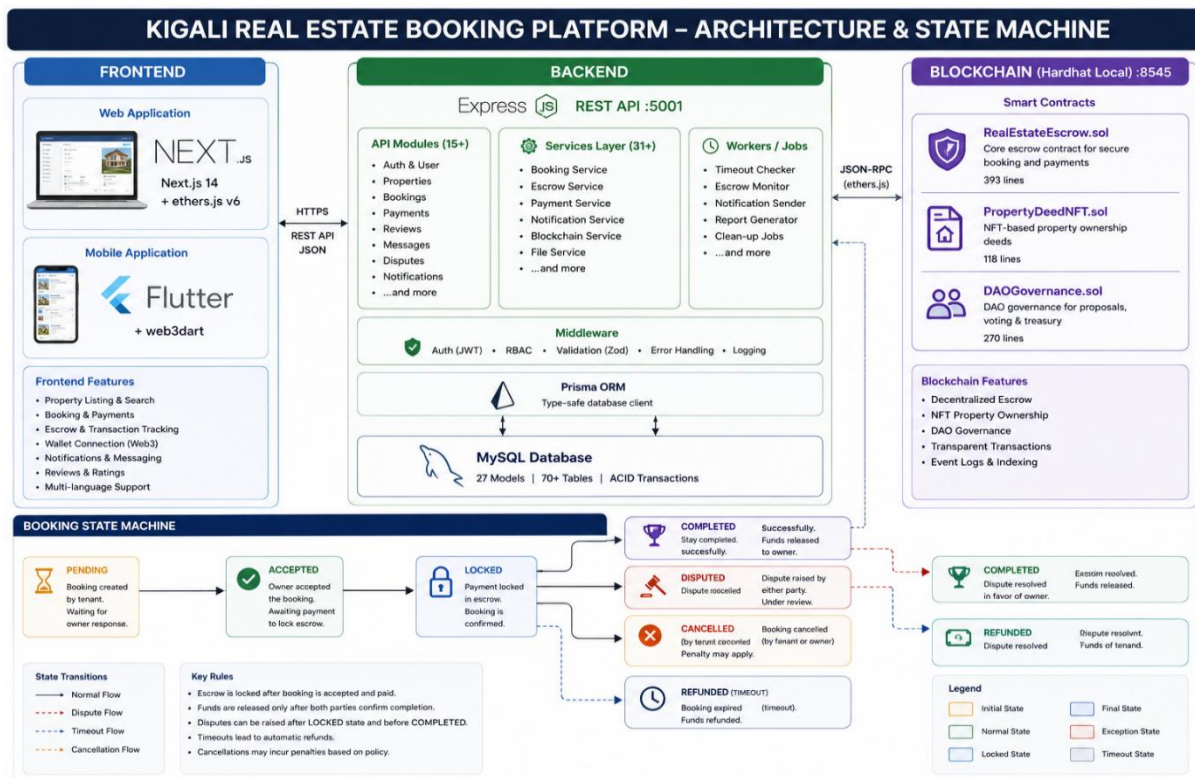


Figure 20: KIGALI REAL ESTATE BOOKING PLATFORM - ARCHITECTURE & STATE MACHINE

Contains several typos: "Dispute resolved" (should be "raised"), "by tenant caconted" (should be "by tenant cancelled"), "Eastim resolved" (should be "Escrow resolved"), "Rispute resolsnt. Funds of tenant." (Should be "Dispute resolved. Funds to tenant."), "tenant" "tenant", etc.

Gas: createBooking ~89K, confirm ~32K, cancel ~28K, timeout ~25K, dispute ~24K, resolve ~30K

## Appendix H: Project Repository

### Project Repository Structure

```
kigali-real-estate-booking/
├── blockchain/
│ ├── contracts/
│ │ ├── RealEstateEscrow.sol # Core escrow (393 lines)
│ │ ├── PropertyDeedNFT.sol # Property NFTs (118 lines)
│ │ └── DAOGovernance.sol # Governance (270 lines)
│ ├── scripts/deploy.ts
│ ├── hardhat.config.ts
│ └── package.json
├── backend/
│ ├── prisma/schema.prisma # 27 models
│ └── src/
```

```

| | |─ index.ts # Express entry point
| | |─ config/ # Env, DB, Swagger
| | |─ middleware/ # Auth, RBAC, validation
| | |─ routes/ # 15 route files
| | |─ services/ # 31 service files
| | |─ schemas/ # Zod validation
| | |─ workers/ # Background jobs
| | └─ package.json
| └─ frontend/
| |─ src/
| | |─ app/ # 32 Next.js pages
| | |─ components/ # 12 React components
| | |─ lib/ # API client, contract ABI
| | |─ context/ # Auth, Wallet, Language
| | |─ hooks/ # useEscrow, useRTL
| | └─ .env.local
| | └─ package.json
| └─ frontend/public/
| |─ manifest.json # PWA manifest (standalone, SVG icons)
| |─ sw.js # Service worker (10KB, generated)
| |─ icons/ # 5 SVG icons for PWA
| └─ EVALUATION_FRAMEWORK.md # UAT methodology + survey instruments
| └─ OBJECTIVES_ASSESSMENT.md # Gap analysis vs. project objectives
| └─ FINAL_REPORT.md # This document
| └─ docker-compose.yml
| └─ package.json # Root workspace

```

Contracts: 3 Solidity | Routes: 15 TS | Models: 27 Prisma

Stack: Next.js 14 + Flutter 3.x + Express + Prisma/MySQL + Solidity/Hardhat

## Semi-Structured Interview Questions

### Section A: Participant Background

1. What is your role in the rental market (tenant, landlord, or broker/agent)?
2. How long have you been involved in renting or managing rental properties in Kigali?
3. Have you previously been involved in a rental deposit agreement? Please explain.

### Section B: Rental Deposit Payment Conditions

4. Under what conditions is a rental deposit usually paid?
5. Who normally receives the deposit (landlord, broker, or another party)?

6. What agreements or evidence are usually provided after making a deposit (receipt, verbal agreement, written contract)?
7. What factors make a tenant confident enough to pay a deposit?

### **Section C: Deposit Withholding Conditions**

8. In your experience, under what circumstances can a rental deposit be withheld?
9. Have you ever experienced or witnessed a dispute related to deposit withholding? What happened?
10. Who normally decides whether a deposit should be withheld or released?
11. What problems commonly occur when handling deposits?

### **Section D: Deposit Refund Conditions**

12. Under what situations should a rental deposit be refunded?
13. How long does it usually take for a deposit to be refunded?
14. Have you ever experienced delays or refusal of a refund? Please explain.
15. What conditions should be met before a deposit is returned?

### **Section E: Smart Contract & System Requirements**

16. What rules do you think a digital rental deposit system should follow to prevent fraud?
17. What conditions should automatically trigger payment release to a landlord?
18. What conditions should automatically trigger a refund to a tenant?
19. How can a system fairly resolve disagreements between landlords and tenants?
20. Would you trust a system that automatically holds and releases deposits based on agreed conditions? Why or why not?